

第3部分. 计算机网络：把世界连起来的魔法

大多数教材读起来都像在嚼干草，对不对？协议、分层、地址、路由……一堆概念砸过来，脑子直接宕机。

不怪你。怪那些教材太无聊了。

在这一部分，你不会读到干巴巴的定义。你会跟小明一起，从零开始搭建一个网络。遇到问题？解决它。概念太抽象？画出来。算不出来？一步一步走。你会亲手算子网掩码、亲自当路由器转发数据包、亲自经历TCP的三次握手和四次挥手。

准备好了吗？把你的大脑调到"接收模式"——我们要开始传输了！

第1章. 计算机网络概论：欢迎来到网络世界

你是否想过，当你打开浏览器输入一个网址，背后发生了什么？几千公里的距离，几十台设备的接力，零点几秒的延迟——这一切都依赖于计算机网络。在这一章，你会看到网络的全貌：它是什么、怎么来的、分几层、跑多快。你会跟小明一起，迈出搭建网络的第一步。

让我们开始！

故事：小明的咖啡店要上网

小明开了一家咖啡店。生意不错，但他发现客人总问同一个问题："老板，Wi-Fi密码多少？"

小明买了路由器，插上电，连上猫。客人能上网了——但小明完全不知道这堆设备在干什么。更糟的是，有一次网络断了，电信说"检查你的物理层"，小明一脸茫然。

"物理层是什么鬼？"小明决定弄明白。

"不如从最基础的问题开始吧。"——老张，小明认识的网络工程师，递给他一本书。

什么是计算机网络？

计算机网络，简单说，就是把一群计算机连起来，让它们能互相通信。

但这定义太干了。我们换个说法：

计算机网络 = 用通信设备和线路，把分布在不同地点的、具有独立功能的计算机系统连接起来，在网络软件的支持下，实现资源共享和数据通信的系统。

拆开来看，三个关键词：

1. 独立功能的计算机 —— 每台机器有自己的CPU、内存，能独立工作。不是终端+主机的哑终端模式。
2. 通信设备和线路 —— 网卡、交换机、路由器、光纤、双绞线……
3. 网络软件支持 —— 协议栈、操作系统网络功能、各种网络应用。

[Diagram: 三台计算机通过交换机连接，每台计算机旁边标注"独立系统"，交换机旁边标注"通信设备"，连接线上标注"通信链路"]

计算机网络是怎么来的？

网络不是一天建成的。咱们从历史长河中捞几个关键节点出来。

第一代：面向终端的网络（1950s-1960s）

那时候计算机贵得要命，只有大型机。多个用户通过终端（只有键盘和显示器，没有处理能力）连接到一台主机。

[Diagram: 一台大型主机在中间，周围辐射出很多终端，标注"终端无处理能力"]

特点：终端没脑子，全靠主机。线路利用率低，主机负担重。

小明点点头："原来以前的'上网'就是远程登录一台大机器啊。"

第二代：计算机网络（1960s-1970s）

ARPANET登场了！1969年，美国国防部高级研究计划局（ARPA）建立了第一个分组交换网络。

关键创新：分组交换（Packet Switching）。数据被切成小块（分组），每个分组独立选路。

[Diagram: ARPANET的四个节点——UCLA、斯坦福研究院、UCSB、犹他大学，通过IMP接口报文处理机连接]

小明眼睛一亮："这不就是我们现在用的互联网的祖先吗？"

没错！ARPANET在1983年全面采用TCP/IP协议，这就是Internet诞生的标志。

第三代：开放式标准化网络（1980s）

各个厂商自己做自己的网络，IBM的SNA、DEC的DECnet.....彼此不通，像个巴别塔。

直到ISO（国际标准化组织）出手，发布了OSI参考模型——一个七层的网络架构标准。虽然OSI协议本身没流行起来，但它的分层思想统治了网络世界至今。

第四代：Internet时代（1990s至今）

1990年代：万维网（WWW）出现，浏览器普及，Internet爆炸式增长。IPv4地址从充裕到紧缺，IPv6逐步推广。移动互联网、物联网、云计算.....网络已经无处不在。

GEEK NOTE: ARPANET在1990年正式关闭，但它的精神活在Internet中。最早的RFC（Request for Comments）至今仍是互联网标准的工作方式——"我们不相信权威，我们相信运行代码和大致共识。"

SHARPEN YOUR PENCIL

把下列网络发展阶段和对应的关键特征连起来。我们已经帮你完成了第一个。

发展阶段	关键特征
面向终端的网络	A. 分组交换，ARPANET
计算机网络	B. WWW爆炸式增长
开放式标准化网络	C. 终端无处理能力，主机集中

发展阶段	关键特征
Internet时代	D. OSI参考模型发布
(答案在下一页)	

SHARPEN YOUR PENCIL SOLUTION

发展阶段	关键特征
面向终端的网络	C. 终端无处理能力，主机集中
计算机网络	A. 分组交换，ARPANET
开放式标准化网络	D. OSI参考模型发布
Internet时代	B. WWW爆炸式增长

计算机网络的分类

网络分类有很多维度，考试最喜欢考两种：按覆盖范围和按拓扑结构。

按覆盖范围分类

类型	缩写	范围	例子
局域网	LAN	10m ~ 几km	办公室、校园、小明咖啡店
城域网	MAN	几km ~ 几十km	一个城市的有线电视网
广域网	WAN	几十km ~ 几千km	Internet、跨国企业专线

小明拍了拍路由器："原来我咖啡店里的网络叫LAN！"

WATCH IT!

考试经常混淆的点：LAN和WAN的区分不在于技术，而在于覆盖范围。同一个以太网技术，用在办公室里是LAN，用在跨城市专线上可能算MAN甚至WAN。

按拓扑结构分类

[Diagram: 六种网络拓扑结构示意图]

1. 总线型 (Bus)：所有节点共享一条通信线路。结构简单，但一个点故障可能导致全网瘫痪。早期的以太网就用这个。
2. 星型 (Star)：所有节点连接到一个中心节点（交换机/集线器）。现代LAN最常用。中心节点坏了全网瘫痪，但单点故障不影响其他。
3. 环型 (Ring)：节点首尾相连形成闭环。令牌环网就是这种。一个节点故障可能导致整个环断裂。
4. 树型 (Tree)：星型的层次化扩展。像一棵倒置的树，根节点是核心交换机。
5. 网状 (Mesh)：节点之间有多条路径。可靠性最高，Internet骨干网就是全网状或部分网状。
6. 混合型 (Hybrid)：实际网络往往是多种拓扑的组合。

BRAIN POWER

小明咖啡店里有10台设备通过一台交换机连接，这是星型拓扑。但如果小明再买一台交换机，把两台交换机连起来，然后各接5台设备——这变成了什么拓扑？为什么实际网络中几乎没有纯粹的总线型拓扑了？

协议与分层：网络的灵魂

小明看着一堆网络设备，头大："这些设备怎么知道彼此在说什么？"

老张笑了："因为它们都说同一种语言——协议（Protocol）。"

什么是协议？

协议 = 规则 + 格式 + 时序

- 语法（格式）：数据怎么组织。比如"前6字节是目的地址，再6字节是源地址....."
- 语义（含义）：每个字段什么意思，收到后该做什么。比如"ACK=1表示确认"
- 同步（时序）：通信的先后顺序。比如"先建立连接，再发数据，最后释放连接"

让小明理解协议的最好方式：写信。

[信封格式示意图]

+-----+	
收信人地址：北京市海淀区中关村1号 张三	<-- 这是"地址格式"（语法）
寄信人地址：上海市浦东新区陆家嘴2号 李四	
邮票：1.2元	<-- 这是"先付钱"（同步）
亲爱的张三：	
好久不见.....	<-- 这是"内容"（语义）
此致	
敬礼	<-- 这是"结束标记"（语法）
+-----+	

你寄信不需要知道邮局内部怎么分拣、用什么车运输、走哪条路——这就是分层的精髓：每一层做好自己的事，对上层屏蔽细节。

分层的核心思想

[Diagram: 分层蛋糕——5层协议栈]

+-----+	
应用层	<-- 小明用微信发消息
+-----+	
运输层	<-- 负责把消息可靠地送到
+-----+	
网络层	<-- 负责找到对方在哪
+-----+	
数据链路层	<-- 负责跳到下一站
+-----+	

```
| 物理层 | <-- 负责把01变成信号
+-----+
```

分层的原则：

- 第N层只使用第N-1层提供的服务
- 第N层向第N+1层提供服务
- 对等层之间遵循相同的协议
- 物理通信只在最底层进行，但逻辑上每层都在和对方的同层"对话"

RELAX

如果刚才那段让你觉得有点绕，别紧张。想象你在寄一封信：你把信交给前台（你是应用层），前台把信放进信封写上地址（运输层），邮局分拣中心决定走哪条路（网络层），卡车司机把信运到下一站（数据链路层），卡车本身在路上跑（物理层）。每一层做好自己的事，你不用管卡车怎么走——这就是分层！

OSI参考模型：七层宝塔

OSI（Open Systems Interconnection）参考模型把网络通信分了七层。记住这个口诀：

"物数网运 会表应" —— 物理层、数据链路层、网络层、运输层、会话层、表示层、应用层

从下往上：

层号	名称	功能	数据单位
7	应用层	为用户应用提供网络服务接口	报文（Message）
6	表示层	数据格式转换、加密解密、压缩	报文
5	会话层	建立、管理、终止会话	报文
4	运输层	端到端可靠传输、流量控制、拥塞控制	报文段（Segment） / 数据报（Datagram）
3	网络层	路由选择、逻辑寻址	分组（Packet） / 数据报
2	数据链路层	成帧、物理寻址、差错检测	帧（Frame）
1	物理层	比特流传输、物理接口定义	比特（Bit）

[Diagram: OSI七层模型——一台主机从上到下封装数据，经过中间系统（路由器只到网络层），再到目标主机从下到上解封装。中间系统旁边标注"路由器只处理下三层"]

小明问："为什么中间的路由器只用到下三层？"

因为路由器只负责把数据包从源地址送到目的地址，它不关心你发的是微信消息还是电子邮件——那是上层的事。就像快递公司不看包裹里装的是书还是衣服。

THERE ARE NO DUMB QUESTIONS

Q: 会话层和运输层都有"连接管理", 它们有什么区别? A: 运输层的连接是端到端的——比如从你电脑到服务器之间建立TCP连接。会话层的"会话"更高级——比如你登录一个网站后, 多次HTTP请求都属于同一个会话。实际上, TCP/IP模型中会话层的功能大多被应用层吸收了。

Q: 为什么表示层要单独分一层? A: 不同计算机可能用不同的数据格式 (比如大端/小端字节序、ASCII/Unicode字符编码)。表示层负责这些格式的转换。但在TCP/IP模型中, 这些功能通常嵌入在应用层协议里 (比如HTTP可以协商字符集)。

Q: OSI七层模型和实际网络一样吗? A: 不完全一样。OSI是参考模型——用来教学和标准化讨论的框架。实际网络中, TCP/IP的四层模型才是王者。但OSI的分层思想影响了所有后来的网络设计。考试两个都要会。

TCP/IP参考模型：四层实干家

如果OSI是"理论家", TCP/IP就是"实干家"。它只有四层, 简洁高效：

[Diagram: TCP/IP四层与OSI七层的对照]

OSI七层	TCP/IP四层
+-----+	+-----+
应用层	应用层
+-----+	(HTTP, DNS, SMTP
表示层	FTP, TELNET...)
+-----+	
会话层	
+-----+	+-----+
运输层	运输层
+-----+	(TCP, UDP)
网络层	+-----+
+-----+	网际层
数据链路层	(IP, ICMP, ARP)
+-----+	+-----+
物理层	网络接口层
+-----+	(以太网, PPP...)
	+-----+

TCP/IP四层功能一览：

1. 网络接口层 (Host-to-Network)：对应OSI的物理层+数据链路层。没有定义具体协议, 只要能传IP数据报就行。以太网、Wi-Fi、PPP都是这一层的具体实现。
2. 网际层 (Internet)：核心是IP协议。提供尽力而为 (best-effort) 的无连接分组传输。ICMP (网络控制消息)、ARP (地址解析)、RARP (反向地址解析) 都在这一层。
3. 运输层 (Transport)：TCP (可靠、面向连接) 和UDP (不可靠、无连接)。提供端到端的通信服务。端口号区分不同的应用进程。
4. 应用层 (Application)：所有高层协议。HTTP、DNS、SMTP、FTP、TELNET、SSH.....

小明总结道："所以TCP/IP把OSI的上三层合并成一个应用层，下两层合并成一个网络接口层？"——完全正确！

GEEK NOTE: OSI协议栈（X.400邮件、X.500目录.....）在市场上失败的原因很简单：太复杂、太慢、太晚。当OSI标准最终定稿时，TCP/IP已经跑遍了全球。互联网工程任务组（IETF）的原则是"We reject kings, presidents and voting. We believe in rough consensus and running code."——先跑起来再说！

网络性能指标：你的网络有多快？

小明测了一下咖啡店的网速："下载速度100Mbps，这个数字到底什么意思？"

速率（Rate）

速率 = 单位时间内传输的比特数，单位是bps（比特/秒，bits per second）。

- 1 Kbps = 10^3 bps（注意：网络速率用10的幂，不是2的幂！）
- 1 Mbps = 10^6 bps
- 1 Gbps = 10^9 bps

WATCH IT!

网络速率是十进制的，但文件大小经常是二进制的。你的100Mbps宽带，实际下载文件的速度最多约12.5 MB/s（100除以8=12.5，因为1Byte=8bit，还有协议开销）。别被运营商忽悠了！

带宽（Bandwidth）

带宽原指信号占用的频率范围（单位Hz），在计算机网络中，带宽通常等价于信道的最大数据传输速率（单位bps）。

"我的带宽是100Mbps" = "我的信道每秒最多能传100M比特"

吞吐量（Throughput）

吞吐量 = 单位时间内实际传输的数据量。

带宽是理论最大值，吞吐量是实际值。就像高速公路限速120km/h（带宽），但堵车时你只能跑30km/h（吞吐量）。

时延（Delay/Latency）

数据从发送到接收经历的总时间。分四种：

1. 发送时延（Transmission Delay）= 数据帧长度 / 发送速率 把数据"推"到链路上需要的时间。数据越大、速率越慢，发送时延越大。
2. 传播时延（Propagation Delay）= 信道长度 / 信号传播速度 信号在介质中传播需要的时间。电信号在铜线中约 $2.3 \cdot 10^8$ m/s，光在光纤中约 $2.0 \cdot 10^8$ m/s。
3. 处理时延（Processing Delay）：路由器检查分组头部、查路由表的时间。
4. 排队时延（Queuing Delay）：分组在路由器队列中等待处理的时间。网络拥塞时，这个时延会剧增。

[Diagram: 四个时延的示意图——发送端：发送时延（数据从网卡推到线路上）；链路上：传播时延（信号沿线路传播）；路由器：排队时延（分组在队列中等待）+ 处理时延（查表转发）；接收端：接收数据]

总时延 = 发送时延 + 传播时延 + 处理时延 + 排队时延

SHARPEN YOUR PENCIL

小明要从北京发一个1000字节的分组到上海。链路速率10Mbps，距离1200km，信号传播速度 2×10^8 m/s。忽略处理时延和排队时延。计算总时延。

发送时延 = _ 传播时延 = _ 总时延 = ____

（答案在下面）

SHARPEN YOUR PENCIL SOLUTION

发送时延 = $1000 \times 8 \text{ bit} / 10 \times 10^6 \text{ bps} = 8000 / 10^7 = 0.8 \text{ ms}$ 传播时延 = $1200 \times 10^3 \text{ m} / 2 \times 10^8 \text{ m/s} = 6 \text{ ms}$ 总时延 = $0.8 + 6 = 6.8 \text{ ms}$

注意！传播时延（6ms）远大于发送时延（0.8ms）。在长距离通信中，传播时延是主导因素——这就是为什么跨洋光纤的延迟很难降低，因为光速是物理极限。

时延带宽积（Delay-Bandwidth Product）

时延带宽积 = 传播时延 * 带宽

这个值表示：在第一个比特到达目的地之前，发送方已经发出去了多少比特。也叫“以比特计的管道长度”。

[Diagram: 一根管道，标注“传播时延”为管道长度，“带宽”为管道粗细。管道内部的容积 = 时延带宽积 = 飞行中的数据量]

小明：“也就是说，卫星链路延迟很大，而且带宽也不小，所以时延带宽积很大——意味着我一次性可以发很多数据出去，但都要等很久才会有回应？”——正确！这就是为什么TCP在卫星链路上需要大窗口。

往返时间（RTT, Round-Trip Time）

RTT = 从发送方发出数据到收到接收方确认的总时间

包括：2 * 传播时延 + 中间节点处理时间 + 接收方处理时间。RTT是TCP超时重传机制的重要参数。

利用率（Utilization）

利用率 = 信道实际使用时间 / 总时间。利用率太高会导致排队时延急剧增加——就像高速公路，车太多就堵死了。

BULLET POINTS

- 计算机网络是把地理位置不同的、具有独立功能的计算机系统通过通信设备和线路连接起来，在网络软件支持下实现资源共享和数据通信的系统。
- 网络发展经历了面向终端的网络 → 计算机网络（ARPANET） → 开放式标准化网络（OSI） → Internet四个阶段。

- 按覆盖范围分为LAN（局域网）、MAN（城域网）、WAN（广域网）；按拓扑分为总线型、星型、环型、树型、网状和混合型。
- 协议定义了通信的语法、语义和同步规则。分层让每层专注于自己的职责，降低复杂度。
- OSI七层模型（物数网运会表应）：物理层、数据链路层、网络层、运输层、会话层、表示层、应用层。
- TCP/IP四层模型：网络接口层、网际层（IP）、运输层（TCP/UDP）、应用层。TCP/IP是实际使用的模型。
- 速率是单位时间传输的比特数（bps）。带宽是信道的最大速率。吞吐量是实际速率。
- 时延 = 发送时延 + 传播时延 + 处理时延 + 排队时延。长距离通信中传播时延通常主导。
- 时延带宽积 = 传播时延 * 带宽，表示管道中"在飞"的最大数据量。
- RTT（往返时间）是发送数据到收到确认的总时间，影响TCP超时设置。
- 利用率过高会导致排队时延剧增，需要权衡吞吐量和时延。

第2章. 物理层：从比特到信号

没有物理层，一切都是空谈。你的微信消息、你的视频流、你的网页请求——最终都要变成电线里的电压变化、光纤中的光脉冲，或者空气中的电磁波。物理层就是那个把0和1变成物理信号的魔术师。

在这一章，小明要学会：信号怎么传、用什么介质传、多路信号怎么共享一条线路、数据交换有几种方式。听起来很硬核？别怕，咱们从一根铜线开始。

故事：小明拉网线

小明想在咖啡店里多拉几条网线，把收银台、办公室和客用Wi-Fi路由器连起来。他去电脑城买网线，发现："超五类？六类？屏蔽？非屏蔽？这是啥？"

更崩溃的是，店员问："你需要多少MHz带宽的？"

小明以为"带宽"就是"网速"，结果店员说的是信号频率范围。原来物理层的"带宽"和应用层的"带宽"是两码事！

老张拍拍小明肩膀："别急，我来教你怎么理解物理层。"

数据通信系统模型

任何数据通信系统都可以抽象成这个模型：

[Diagram: 数据通信系统模型——信源（发送方）→ 发送器（调制器）→ 信道（传输介质）→ 接收器（解调器）→ 信宿（接收方）。信道上标注"噪声源"，箭头指向信道]

- 信源：产生数据（你的电脑）
- 发送器：把数据编码/调制成适合信道传输的信号（网卡、调制解调器）
- 信道：信号的传输通道（双绞线、光纤、空气.....）
- 接收器：把信号解码/解调还原为数据
- 信宿：数据的目的地（服务器）
- 噪声源：信道中干扰信号的来源（电磁干扰、串扰.....）

小明点点头："所以网卡其实同时是发送器和接收器，一条网线是信道，旁边的微波炉是噪声源？"

完全正确！微波炉的电磁波可能干扰Wi-Fi，因为它和2.4GHz Wi-Fi频段重叠。这就是为什么你的Wi-Fi在微波炉工作时有时候会卡。

模拟信号与数字信号

信号分两种：

模拟信号：连续变化。像收音机的FM信号——波形平滑连续。参数（幅度、频率、相位）可以取无限多个值。

数字信号：离散变化。只有有限的几个状态（通常是两个：0和1）。方波就是典型的数字信号。

[Diagram: 左边是模拟信号（正弦波，平滑曲线），右边是数字信号（方波，突然跳变）]

计算机产生的是数字信号（0101.....），但有些信道（比如电话线）只能传模拟信号。怎么办？调制——把数字信号变成模拟信号。调制解调器（Modem）就干这个。

GEEK NOTE: 为什么叫"Modem"？它是Modulator（调制器）+ Demodulator（解调器）的组合。发送时把数字调成模拟，接收时把模拟解调回数字。你家那个"猫"（宽带猫），就是干了半个多世纪前电话线上网Modem类似的活——只是技术从电话拨号升级到了光纤/xDSL。

信道的极限容量：两个定理统治物理层

信道的速率不可能无限大。奈奎斯特和香农用两个定理划定了物理界限。

奈奎斯特定理（Nyquist, 1924）

用于无噪声信道：

最大数据传输速率 = $2W * \log_2(V)$ (bps)

- W = 信道带宽（Hz）
- V = 信号离散等级数（二进制时 $V=2$ ， $\log_2(2)=1$ ）

小明掏出计算器："如果信道带宽是3kHz，只用二进制信号（ $V=2$ ），那最大速率 = $2 * 3000 * 1 = 6000$ bps。"——对！这就是为什么早期电话线上网最快到56Kbps都算奇迹。

SHARPEN YOUR PENCIL

一个无噪声信道带宽为20kHz，使用4级（四进制）信号。问理论最大数据传输速率是多少？

最大速率 = $2W * \log_2(V)$ = _

（答案在下面）

SHARPEN YOUR PENCIL SOLUTION

最大速率 = $2 * 20000 * \log_2(4) = 40000 * 2 = 80,000$ bps = 80 Kbps

信号等级每翻一倍（ V 从2→4→8→16.....），速率就增加 $\log_2(V)$ 倍。但等级越多，信号越容易受噪声干扰——这就是为什么不能无限增加 V 。

香农定理（Shannon, 1948）

用于有噪声信道（这才是现实世界）：

最大数据传输速率 = $W * \log_2(1 + S/N)$ (bps)

- W = 信道带宽 (Hz)
- S/N = 信噪比 (信号功率与噪声功率之比)

信噪比常用分贝 (dB) 表示： $SNR = 10 * \log_{10}(S/N)$ dB

关键结论：噪声的存在给信道速率设了一个天花板，无论你怎么编码都无法突破。

小明："所以如果信噪比是30dB (即 $S/N=1000$)，带宽3kHz，那最大速率 = $3000 * \log_2(1001) \approx 3000 * 10 \approx 30000$ bps ≈ 30 Kbps。" —— 漂亮！这就是有噪声信道的香农极限。

WATCH IT!

奈奎斯特定理告诉你：增加信号等级可以提高速率。香农定理告诉你：增加信号等级会让信号更容易被噪声淹没。两个定理一起使用时，取两个公式计算结果的较小值作为实际极限。

调制与编码

数字数据要通过模拟信道传输？用调制。数字数据要通过数字信道传输？用编码。

数字 → 模拟：调制技术

1. 调幅 (ASK, Amplitude Shift Keying)：用不同幅度表示0和1。比如高幅度=1，低幅度=0。简单但易受干扰。
2. 调频 (FSK, Frequency Shift Keying)：用不同频率表示0和1。比如高频=1，低频=0。抗干扰能力强。
3. 调相 (PSK, Phase Shift Keying)：用不同相位表示0和1。比如相位 $0^\circ=1$ ，相位 $180^\circ=0$ 。更高级的QPSK (四相) 一次可以传2个比特。
4. 正交幅度调制 (QAM)：幅度+相位一起调，频谱效率最高。比如16-QAM，一次可以传4个比特。

[Diagram: ASK、FSK、PSK、QAM四种调制方式的波形对比图，标注每种方案的特点]

数字 → 数字：编码技术

1. 不归零编码 (NRZ)：高电平=1，低电平=0。简单但有同步问题——长串0或1会让接收方丢失时钟。
2. 曼彻斯特编码：每个比特中间有一次跳变。上升沿=0，下降沿=1。自带时钟同步，但带宽利用率只有50%。
3. 差分曼彻斯特编码：比特开始处有跳变=0，无跳变=1，中间始终有跳变。以太网10BASE-T用的就是这个。

[Diagram: 同一段比特流 (10110010) 在NRZ、曼彻斯特、差分曼彻斯特编码下的波形对比]

传输介质：信号跑的"路"

双绞线 (Twisted Pair)

两根铜线绞在一起，绞合可以减少电磁干扰。以太网最常用的介质。

类别	带宽	应用
Cat5（五类）	100MHz	100BASE-TX（100Mbps以太网）
Cat5e（超五类）	100MHz	1000BASE-T（千兆以太网）
Cat6（六类）	250MHz	1000BASE-T（更稳定）
Cat6a	500MHz	10GBASE-T（万兆以太网）

分屏蔽双绞线（STP）和非屏蔽双绞线（UTP）。咖啡店、办公室、家用绝大多数是UTP——便宜、好装。

同轴电缆（Coaxial Cable）

中心铜芯→绝缘层→金属屏蔽网→外皮。抗干扰能力强，早期以太网（10BASE5/10BASE2）和有线电视用。现在基本被双绞线+光纤取代。

光纤（Optical Fiber）

光在玻璃丝中全反射传播。带宽极大、距离极远、抗电磁干扰。

- 单模光纤（SMF）：纤芯极细（约9μm），只传一种模式的光。使用激光，适合远距离（几十到上百公里）。
- 多模光纤（MMF）：纤芯较粗（50/62.5μm），多模式光同时传。使用LED，适合短距离（几百米到几公里）。

[Diagram: 单模光纤vs多模光纤的截面图和光传播路径。单模：细芯，光线直直走；多模：粗芯，多条光线以不同角度反射前进，产生模间色散]

小明问："光纤快还是铜线快？" —— 信号在光纤中的传播速度（约 2.0×10^8 m/s）略低于铜线（约 2.3×10^8 m/s），但光纤的带宽远超铜线，所以单位时间能传的数据多得多。

无线传输

没有物理线缆，信号通过电磁波在空气中传播。

方式	频率范围	特点	应用
无线电波	3kHz-1GHz	穿透力强，全向传播	广播、Wi-Fi（2.4/5GHz）
微波	1GHz-300GHz	直线传播，定向	卫星通信、微波接力
红外线	300GHz-400THz	不能穿透墙壁，短距	遥控器、红外数据传输
激光	更高	极定向，受天气影响	激光通信（极少用于通用网络）

信道复用技术：一条路跑多辆车

如果小明只有一条光纤，但想同时上网和打电话，怎么办？复用——在同一条物理链路上同时传输多路信号。

频分复用（FDM，Frequency Division Multiplexing）

把信道带宽按频率分成多个子信道，每路信号占一个频段。就像广播电台——每个台占一个频率，互不干扰。

[Diagram: 频分复用示意图——横轴是频率，不同频率段分配给不同信号，之间有保护频带]

典型应用：有线电视、ADSL（电话线上网和通话同时进行）、无线电广播。

时分复用（TDM，Time Division Multiplexing）

把时间分成固定长度的时间片（时隙），轮流给各用户使用。

[Diagram: 时分复用示意图——横轴是时间，轮流分配给用户A、B、C、D。每个用户在自己的时隙内独占全部带宽]

同步TDM（STDM）：即使某个用户没数据要发，它的时隙也浪费在那里（空转）。适用于恒定速率业务。

异步TDM（统计TDM/ATDM）：按需分配时隙，有数据的用户才能用时隙。效率更高，适用于突发性数据。

波分复用（WDM，Wavelength Division Multiplexing）

光纤上的FDM——不同波长的光走同一条光纤。

[Diagram: 波分复用示意图——多个不同颜色的激光束通过合波器合并到一根光纤，在远端通过分波器分开]

DWDM（密集波分复用）可以在一根光纤上复用几十甚至上百个波长，单光纤传输容量达到Tbps级别——这是骨干网的核心技术。

码分复用（CDM，Code Division Multiplexing）

CDMA（码分多址）：所有用户在同一时间、同一频率传输，但用不同的“码片序列”区分。

每个用户分配一个唯一的码片序列（chip sequence），接收方用该码片序列做内积，分离出该用户的信号，其他用户的信号变成噪声。

这就像在一个大房间里，每对人用不同的语言交谈——你只听懂汉语，英语对你来说只是背景噪音。

BRAIN POWER

在CDMA中，如果所有用户的信号功率都一样，但用户数量越来越多，会发生什么？为什么CDMA系统有一个“软容量”的概念？（提示：想想香农定理和信噪比的关系）

数据交换方式：数据怎么从A到B

小明要在上海（小明出差了）把他咖啡店的监控视频传回北京的办公室。数据怎么过网络？有几种交换方式。

电路交换（Circuit Switching）

打电话的方式。先建立一条专用物理连接，然后通话，最后释放连接。三个步骤：建立连接→数据传输→释放连接。

特点：独占信道，延迟稳定，适合实时通信（电话）。但信道利用率低——通话中很多时间是沉默的，线路却一直被占用。

报文交换（Message Switching）

把整个报文（一整封"信"）从源节点传到中间节点，中间节点存储→检查→转发。存储转发（store-and-forward）。

特点：不需要事先建立连接，线路利用率高。但延迟大（每个节点都要等整个报文收完再转发），不适合实时通信。几乎已经被分组交换取代。

分组交换（Packet Switching）

把报文切成一个个小块（分组/数据包），每个分组独立选路传输。

这是现在Internet的基础！ARPANET之所以伟大，就是因为它率先采用了分组交换。

分组交换又分两种模式：

1. 数据报（Datagram）方式

每个分组携带完整的源/目的地址，独立选路，可能走不同路径，可能乱序到达。IP协议就是数据报方式。

特点：无连接、灵活、容错性强（某条路断了，分组可以走别的路）。

2. 虚电路（Virtual Circuit）方式

发送数据前先建立一条"虚"的路径（不是独占的物理电路，而是逻辑上的路径），所有分组沿同一路径传输。类似电路交换和分组交换的混合体。

三个阶段：虚电路建立 → 数据传输 → 虚电路释放。

[Diagram: 数据报方式——三个分组走了三条不同的路径，到达顺序可能乱；虚电路方式——所有分组沿同一条路径走]

三种交换方式比较

特性	电路交换	报文交换	分组交换
连接建立	需要	不需要	不需要（数据报）/ 需要（虚电路）
传输单位	比特流	整个报文	分组
专用路径	有	无	无
转发方式	直接转发	存储转发	存储转发
延迟	最小	最大	中等
灵活性	差	好	最好
典型应用	电话网	电报	Internet

SHARPEN YOUR PENCIL

小明要传一个10MB的文件（1MB=10⁶字节），网络有三段链路，每段链路速率都是1Mbps。忽略传播时延和处理时延。分别计算使用以下方式传输的总时间：

(a) 电路交换：先花1秒建立连接，然后传输。(b) 报文交换：整个报文作为整体传输。(c) 分组交换：文件分成10000字节一个分组，每个分组独立转发。

(提示：分组交换中，因为分组很小，第一个分组开始在第二段传输时，第二个分组还在第一段传输——这就是"流水线"效果。)

(a) 总时间 = __ (b) 总时间 = __ (c) 总时间 = __

(答案在下面)

SHARPEN YOUR PENCIL SOLUTION

文件大小： $10 \times 10^6 \times 8 = 80 \text{ Mbit}$ 每段链路传输时间（单段）： $80 \text{ Mbit} / 1 \text{ Mbps} = 80 \text{ 秒}$

(a) 电路交换：建立连接1秒 + 传输80秒 = 81秒（三段链路是一条通路，传输时间只需要一段）

(b) 报文交换：整报文的存储转发，每个节点等报文收完再转发。总时间 = $80 \times 3 = 240 \text{ 秒}$

(c) 分组交换：共有 $80 \times 10^6 / (100008) = 1000$ 个分组。每个分组的单段传输时间 = $10000 \times 8 / 10^6 = 0.08 \text{ 秒}$ 。第一个分组到达： $0.08 \times 3 = 0.24 \text{ 秒}$ （经过三段链路）。此后每0.08秒到达一个分组。最后一个分组到达 = $0.24 + (1000-1) \times 0.08 = 0.24 + 79.92 = 80.16 \text{ 秒}$

分组交换完胜！因为分组小的"流水线"效应，总时间接近电路交换，但它不需要建立连接，共享链路更高效。这就是为什么Internet用分组交换而不是电路交换！

BULLET POINTS

- 数据通信系统包括信源、发送器、信道、接收器、信宿和噪声源。物理层把比特流（0和1）变成物理信号在介质中传输。
- 模拟信号连续变化，数字信号离散变化。计算机用数字信号，需要通过调制/编码适配不同信道。
- 奈奎斯特定理（无噪声）：最大速率 = $2W \log_2(V)$ 。香农定理（有噪声）：最大速率 = $W \log_2(1+S/N)$ 。两者取较小值为实际极限。
- 调制技术：ASK（调幅）、FSK（调频）、PSK（调相）、QAM（正交幅度调制）。编码技术：NRZ、曼彻斯特、差分曼彻斯特。
- 传输介质：双绞线（最常用，UTP/STP，Cat5e/6/6a）、同轴电缆（有线电视）、光纤（单模远距/多模短距，带宽极大）、无线（无线电、微波、红外、激光）。
- 信道复用：FDM（频分，共享频率）、TDM（时分，共享时间，分同步和异步）、WDM（波分，光纤上不同波长）、CDM/CDMA（码分，同一时间同一频率不同码片序列）。
- 数据交换方式：电路交换（先建连接，独占资源）、报文交换（存储转发整报文）、分组交换（存储转发分组，包括数据报方式和虚电路方式）。
- 分组交换是Internet的基础——因为"流水线"效应和共享链路，总延迟接近电路交换，但利用率更高，更灵活。

第3章. 数据链路层：可靠的一跳

物理层只是把比特扔到线上去——至于这些比特有没有送到、有没有送错，物理层才不管呢。这就好比你把一封信扔进邮筒，但不贴邮票、不写地址——指望邮递员猜？

数据链路层就是那个认真负责的一层：它把比特组装成帧、检查有没有出错、控制发送速度、确保每一"跳"（hop）都可靠。在这一章，小明要变成一位"帧"的工匠。

故事：小明发现Wi-Fi丢包

小明咖啡店的收银台通过Wi-Fi连接路由器，但有时候POS机打印出来的小票缺行，或者显示乱码。

"数据丢了？还是传错了？"小明用ping测试，发现有5%的丢包率。

老张说："别只看网络层，问题出在数据链路层。Wi-Fi的信号在空中容易受干扰，数据链路层有差错检测机制来发现这些错误——但如果干扰太严重，帧就被丢弃了。移动一下路由器位置，问题就解决了。"

小明照做，丢包率降到0%。但他还不满足："差错检测具体是怎么做的？如果错了怎么办？会不会重传？"

数据链路层的基本角色

数据链路层在物理层之上，负责把不可靠的物理链路变成可靠的逻辑链路。

三大核心任务：

1. 成帧 (Framing)：把物理层传来的比特流切分成一个个帧，加上帧头和帧尾。
2. 差错检测 (Error Detection)：检测帧在传输中是否出错（出错了就丢弃或要求重传）。
3. 流量控制 (Flow Control)：让发送方别发太快，接收方处理不过来。

[Diagram: 数据链路层在发送方把网络层的数据报封装成帧（加上帧首部和帧尾部），在接收方拆帧还原数据报。帧首部包含地址和控制信息，帧尾部包含差错检测码]

RELAX

如果你觉得"学完TCP还要学数据链路层的流量控制？"——别紧张。数据链路层的流量控制只控制点到点（一跳）的速度，而TCP控制的是端到端。范围不同，但思路相似——都是"别发太快"。

成帧方法

比特流是一串0和1，怎么知道哪是帧的开头、哪是帧的结尾？

方法一：字符计数法

在帧头放一个字段，标明帧中有多少个字符。

```
帧： | 5 | H | E | L | L | O | 3 | W | O | R | ...
```

第一帧：长度=5，读5个字符"HELLO"。下一帧长度=3.....

问题：如果计数字段在传输中出错，后面全部乱掉，而且无法恢复。

方法二：字符填充法

用特殊字符标记帧的开始和结束。如DLE STX（数据链路转义+正文开始）→ 数据 → DLE ETX（转义+正文结束）。

问题：如果数据中恰好出现DLE怎么办？填充——在数据的DLE前面再加一个DLE。接收方看到两个连续的DLE就去掉一个。

方法三：比特填充法（HDLC/PPP用的就是它！）

用比特模式 01111110（0x7E）作为帧的开始和结束标志。

问题：数据中如果出现连续的6个1（01111110），接收方会误以为是帧尾。解决：发送方检查数据部分，每遇到5个连续的1就自动塞入一个0。接收方看到5个1后面的0就自动移除。

这叫零比特填充（Bit Stuffing）：

```
原始数据: ...01111110... (数据中有6个1, 即01111110)
发送方填0: ...011111010... (5个1后塞入一个0)
接收方去0: ...01111110... (去掉塞入的0, 还原)
```

[Diagram: 比特填充示意——发送方扫描数据流，每5个1后插入0；接收方扫描数据流，每5个1后删除0。帧标志01111110不会被误识别]

小明惊呼："所以整个帧里，除了开头和结尾的01111110，绝对不会出现连续的6个1！因为发送方把任何6个1都拆成了5+1。太聪明了！"

SHARPEN YOUR PENCIL

原始数据比特流（不包括帧标志）为：1111101111110111111001

经过零比特填充后，发送方实际发送的数据是（请写出）：

（答案在下面）

SHARPEN YOUR PENCIL SOLUTION

原始数据: 11111 0 11111 10 11111 1001

第一组：11111 后是 0，不处理 → 111110 第二组：11111 后是 1，需填充0 → 1111101 第三组：11111 后是 1（11111后跟1001，第一个是1），需填充0 → 1111100...

仔细分析：数据中有三个"5个连续1"的位置：位置1: 11111 后接0 → 111110 位置2: 11111 后接1 → 111110 (在这5个1后面插入0) 变成 1111101 位置3: 11111 后接1 → 111110 (插入0) 变成 1111100...

填充后：111110 1111101 0 111110 001 即：11111011111010111110001

差错检测

数据在传输过程中可能会出错：0变1，1变0。怎么发现？

奇偶检验（Parity Check）

奇偶检验码：在数据后面加一个检验位，使得整个串中1的个数为奇数（奇检验）或偶数（偶检验）。

- 数据：1011001（4个1）
- 偶检验位：0（4+0=4，偶数）→ 发送 1011001_0
- 奇检验位：1（4+1=5，奇数）→ 发送 1011001_1

问题：只能检出奇数个比特错误。如果同时两个比特翻错，奇偶性不变，检不出。

二维奇偶检验：把数据排成矩阵，行和列各自做奇偶检验。可以检错+定位+纠错一位。

CRC循环冗余检验（重点！考试必考！）

CRC是数据链路层最常用的差错检测方法。以太网帧的FCS（帧检验序列）用的就是CRC-32。

CRC的原理：

把数据看作一个二进制多项式 $M(x)$ 。使用一个双方约定的生成多项式 $G(x)$ 。在数据后面附加一个校验和（余数 R ），使得整个帧的多项式能被 $G(x)$ 整除。

接收方用同样的 $G(x)$ 去除收到的帧——如果能整除（余数为0），传输正确；否则有错。

CRC计算步骤（模2运算）：

1. 设数据 M 有 k 位，生成多项式 $G(x)$ 的阶为 r （即 $r+1$ 位二进制）。
2. 在数据 M 后面追加 r 个0，得到 $M * 2^r$ （ $k+r$ 位）。
3. 用 $M * 2^r$ 模2除以 G （ $r+1$ 位），得到 r 位余数 R 。
4. R 就是CRC校验码。发送方发送：数据 M + 校验码 R 。
5. 接收方计算：收到的 $(k+r)$ 位除以 G 的余数。余数为0则无错。

SHARPEN YOUR PENCIL

数据 $M = 101001$ （ $k=6$ 位），生成多项式 $G(x) = x^3 + x^2 + 1$ （即 $G=1101$ ， $r=3$ ）。

请逐步计算CRC校验码 R ，并写出最终发送的帧。

第一步： M 后面追加 r 个0 → _ 第二步：模2除法（逐位异或） → _ 第三步：余数 $R =$ _ 最终发送帧 = _

（答案在下一页）

SHARPEN YOUR PENCIL SOLUTION

$M = 101001$, $G = 1101$, $r = 3$

第一步： M 后追加3个0 → 101001000

第二步：模2除法：

```

1101 ) 101001000
      1101          (第一位1, 商1, 异或: 1010 xor 1101 = 0111)
      -----
        1110        (第一位0, 商0, 拉下一位)
        1101        (第一位1, 商1, 异或: 1110 xor 1101 = 0011)
        -----
          0111      (第一位0, 商0)
          0000
          -----
            1110    (第一位1, 商1, 异或: 1110 xor 1101 = 0011)
            1101
            -----
              
```

```

      0110 (第一位0, 商0)
      0000
      -----
      1100 (第一位1, 商1, 异或: 1100 xor 1101 = 0001)
      1101
      -----
      001 = 余数R

```

余数R = 001

最终发送帧 = 101001 001 (数据M + 余数R)

验证：接收方计算 101001001 / 1101 的余数，应该为 000 (整除)。

模2除法的核心就是异或 (XOR)：

- 被除数最高位是1 → 商1，用除数异或
- 被除数最高位是0 → 商0，什么也不做
- 每次异或后去掉最高位，拉下被除数的下一位
- 直到被除数全部处理完

常用标准生成多项式：

名称	G(x)	应用
CRC-16	$x^{16}+x^{15}+x^2+1$	HDLC
CRC-CCITT	$x^{16}+x^{12}+x^5+1$	通信协议
CRC-32	$x^{32}+x^{26}+x^{23}+...+1$	以太网

流量控制与可靠传输协议

数据链路层不仅要检错，还要可靠传输。这意味着：

1. 出错的帧要重传
2. 发送方不能太快，接收方要能处理
3. 帧不能重复、不能丢失

从简单到复杂，我们一步步来。

停止等待协议 (Stop-and-Wait)

最朴素的可靠传输方式：发一个帧，等确认，再发下一个。

[Diagram: 停止等待协议时序图——发送方发帧1 → 接收方收到 → 接收方回ACK → 发送方收到ACK → 发送方发帧2 →]

基本流程：

1. 发送方发一帧，启动超时定时器
2. 接收方收到后回ACK

3. 发送方收到ACK，发下一帧
4. 如果超时没收到ACK，重传该帧

帧编号：为了避免重复帧混乱，用1位序号（0和1交替）。接收方根据序号判断是新的帧还是重传。

信道利用率 = (帧传输时间) / (帧传输时间 + RTT)

如果传播时延很大，大部分时间在等ACK——利用率极低。

SHARPEN YOUR PENCIL

小明用停止等待协议通过卫星链路传输数据。帧长4000bit，链路速率50Kbps，RTT=500ms。求信道利用率。

帧传输时间 = _ 信道利用率 = _

SHARPEN YOUR PENCIL SOLUTION

帧传输时间 = 4000 bit / 50 Kbps = 4000 / 50000 = 0.08秒 = 80ms 信道利用率 = 80 / (80 + 500) = 80 / 580 ≈ 13.8%

恐怖吧！86.2%的时间在干等。卫星链路上用停等协议简直是浪费。这就是为什么需要连续ARQ。

连续ARQ协议 —— 后退N帧（Go-Back-N）

发送方维护一个发送窗口，可以连续发送窗口内的所有帧，不需要等待确认。

[Diagram: Go-Back-N协议示意图——发送方窗口大小为W=4，连续发出帧0,1,2,3。接收方返回ACK0,ACK1,ACK2丢失。发送方超时，从丢失的帧2开始，重传帧2,3,4,5]

接收方采用累积确认：ACK n 表示"n号帧及之前的所有帧都已正确收到"。

出错处理：如果帧m出错，接收方丢弃帧m及之后所有帧（不再缓存）。发送方超时后从帧m开始重传所有后续帧。

发送窗口大小 $W \leq 2^n - 1$ （n为序号比特数，如n=3则W≤7）。

优点：简单，接收方不需要缓冲区。缺点：一旦出错，要重传很多可能已经正确到达的帧——浪费带宽。

选择重传ARQ（Selective Repeat）

只重传出错的帧，正确收到的帧先存起来。

[Diagram: 选择重传ARQ——帧2出错，接收方缓存后续正确收到的帧3,4,5，只请求重传帧2。收到帧2后，按序交付]

接收方有独立的确认：NAK 2（或SACK）明确请求重传帧2。

发送窗口 $W \leq 2^{(n-1)}$ （更严格，为了避免序号混淆）。

优点：效率高，只重传出错帧。缺点：接收方需要更大的缓冲区来重排序。

三种ARQ对比

特性	停等	GBN	选择重传
窗口大小(发送)	1	$\leq 2^{n-1}$	$\leq 2^{n-1}$
窗口大小(接收)	1	1	> 1
确认方式	单个	累积	单个/SACK
出错重传	该帧	该帧及之后	仅出错帧
接收方缓存	无	无	需要
效率	低	中	高

BE THE ROUTER / 模拟数据链路层

假设你是接收方，使用Go-Back-N协议，窗口大小 $W=4$ ，序号用3位（0-7）。发送方按序发送帧0,1,2,3。但帧1在传输中丢失，帧0,2,3正确到达。请写出你的行为：

1. 收到帧0后，你做什么？___
2. 帧1丢失（没收到），帧2到达，你做什么？___
3. 帧3到达，你做什么？___
4. 发送方超时后，重传了哪些帧？___
5. 你收到重传帧后，向网络层交付了哪些帧？___

BE THE ROUTER 答案

1. 收到帧0，正常。交付网络层。回ACK=0（期待帧1）。
2. 帧2到达，但期待帧1。丢弃帧2（GBN不缓存乱序帧）。回ACK=0（仍在等帧1）。
3. 帧3到达，仍期待帧1。丢弃帧3。回ACK=0。
4. 发送方超时，从帧1开始重传：重传帧1,2,3。
5. 收到重传的帧1 → 交付 → ACK=1 → 帧2 → 交付 → ACK=2 → 帧3 → 交付 → ACK=3。

虽然帧2,3第一次正确到达，但被丢弃了。这就是Go-Back-N的"冤枉"重传。

点对点协议PPP

小明要在家通过ADSL上网——他的电脑和ISP的接入服务器之间用的就是PPP（Point-to-Point Protocol）。

PPP的特点

- 简单：不提供可靠传输（序号和确认），只做最基本的成帧和差错检测
- 支持多种网络层协议：可以同时承载IP、IPX等多种网络层协议
- 支持多种链路层协议：可以在各种串行线路上运行
- 支持身份认证：PAP（口令认证）和CHAP（挑战握手认证）

PPP帧格式

```

+-----+-----+-----+-----+-----+-----+
| Flag   | Address | Control | 协议   | 数据   | FCS    | Flag   |

```

```
| 01111110 | 11111111 | 00000011 | 2字节 | ≤1500B | 2/4字节 | 01111110 |
+-----+-----+-----+-----+-----+-----+
```

- Flag（标志）：0x7E（01111110），帧的开始和结束
- Address：固定0xFF（广播地址，PPP是点对点的，不需要地址）
- Control：固定0x03
- 协议：标识上层协议（0x0021=IP，0xC021=LCP，0x8021=NCP.....）
- FCS：CRC校验

PPP使用比特填充法来解决数据和Flag冲突的问题。

PPP链路建立过程

[Diagram: PPP链路状态图——Dead（静止）→ Establish（建立，LCP协商）→ Authenticate（验证，可选PAP/CHAP）→ Network（网络，NCP协商IP等）→ Terminate（终止）→ Dead]

1. LCP（链路控制协议）：建立、配置和测试数据链路连接。协商MRU（最大接收单元）、认证方式等。
2. 认证（可选）：
 - PAP（口令认证协议）：客户端明文发送用户名+密码。不安全，已过时。
 - CHAP（挑战握手认证协议）：服务器发挑战值 → 客户端用密码+挑战值计算散列 → 服务器验证。密码不通过网络传输，更安全。
3. NCP（网络控制协议）：为网络层协议分配IP地址等参数。对于IP，就是IPCP（IP控制协议）。

介质访问控制（MAC）子层

到目前为止，我们讨论的都是点对点链路——只有两端的通信。但Wi-Fi呢？以太网呢？一条总线上接了N多台设备，谁先发？谁什么时候发？

这就是介质访问控制（Medium Access Control）要解决的问题——多台设备如何共享同一条信道。

ALOHA协议

夏威夷大学的ALOHA协议是MAC协议的鼻祖。

纯ALOHA（Pure ALOHA）：想发就发，冲突了再说。如果冲突（两个节点同时发），各自随机等待一段时间后重传。信道利用率上限：18.4%。

时隙ALOHA（Slotted ALOHA）：把时间分成等长时隙，只能在时隙开始时发送。冲突窗口减半。信道利用率上限：36.8%。

CSMA协议（载波监听多路访问）

ALOHA不“听”信道，直接发。CSMA聪明多了——先听再发。

1-坚持CSMA：

1. 持续监听信道
2. 信道空闲 → 立即发送（概率=1，所以叫“1-坚持”）

3. 信道忙 → 一直等到空闲，然后立即发送
4. 如果冲突 → 等待随机时间后重试

问题：如果多个节点同时在等信道空闲，信道一变空闲大家全发出去——必然冲突。

非坚持CSMA：

1. 监听信道
2. 信道空闲 → 立即发送
3. 信道忙 → 不等了，随机退避一段时间后再监听

优点：减少了冲突概率。缺点：信道可能空闲着没人用（都在退避呢）。

p-坚持CSMA：

1. 监听信道
2. 信道空闲 → 以概率p发送，以概率(1-p)推迟到下一个时隙
3. 信道忙 → 等下一个时隙再监听

p-坚持是1-坚持和非坚持的折中，适用于时隙信道。

CSMA/CD —— 以太网的核心

CSMA/CD = CSMA + Collision Detection（冲突检测）

以太网的灵魂就是CSMA/CD！工作流程：

1. 先听后发：发之前监听信道，空闲才发
2. 边发边听：发送过程中也在监听
3. 冲突则停：一旦检测到冲突，立即停止发送
4. 退避重发：等待随机时间后重试

冲突检测：以太网节点在发送时检测信道上电压是否异常升高（两个信号叠加）。最短帧长必须保证在发送完之前能检测到冲突——所以以太网有最小帧长64字节和最大传输距离的限制。

SHARPEN YOUR PENCIL

在10Mbps以太网中，假设信号传播速度 2×10^8 m/s，两个最远节点之间的距离为2500m。问：最小帧长应该是多少字节，才能确保节点在发送完之前检测到冲突？

往返传播时延（冲突窗口）= __ 最小帧传输时间 $\geq$ __ 最小帧长（bit）= __ 最小帧长（字节）= __

SHARPEN YOUR PENCIL SOLUTION

往返传播时延 = $(2500 \times 2) / (2 \times 10^8) = 5000 / 2 \times 10^8 = 25 \times 10^{-6}$ 秒 = 25 μ s

这就是"冲突窗口"——在最坏情况下，发送方需要25 μ s才能检测到冲突。

最小帧传输时间 $\geq$ 冲突窗口 = 25 μ s

最小帧长（bit）= 10 Mbps $\times$ 25 μ s = $10 \times 10^6 \times 25 \times 10^{-6}$ = 250 bit

最小帧长（字节）= 250 / 8 = 31.25 → 取64字节

标准以太网规定最小帧长64字节，比理论最小值大了一倍多——加了安全余量。

截断二进制指数退避算法

冲突后什么时候重试？不是随机的——以太网用截断二进制指数退避：

1. 确定退避时间槽（对于10Mbps以太网，槽=51.2μs）
2. 第k次冲突后（ $k \leq 10$ ），从 $[0, 2^k - 1]$ 中随机选一个数r
3. 等待 $r \times$ 时间槽 后重传
4. 第16次冲突后仍失败，丢弃该帧，向上层报错

退避范围随冲突次数指数增长——冲突越多等越久，防止网络恶化。

CSMA/CA —— Wi-Fi的核心

为什么Wi-Fi不用CSMA/CD？

因为无线信号衰减太快，发送方在发送时无法同时监听（自己发的信号太强，盖过了接收到的信号），所以没法做冲突检测。而且存在隐藏站问题和暴露站问题。

CSMA/CA（Collision Avoidance，冲突避免）的策略：

1. 先监听（物理载波监听 + 虚拟载波监听/NAV）
2. 信道空闲 → 等DIFS → 发RTS（请求发送）
3. 接收方回CTS（允许发送）——周围节点听到后设置NAV，不发送
4. 发数据 → 接收方回ACK
5. 如果没收到ACK → 冲突，重传

[Diagram: CSMA/CA的RTS/CTS/DATA/ACK交互时序图——标注DIFS、SIFS、NAV的作用范围]

隐藏站问题：A能听到B，C也能听到B，但A和C互相听不到。A和C可能同时给B发数据，在B处冲突。RTS/CTS就是解决方案——B发CTS时A和C都听到。

以太网：从10M到10G的传奇

以太网（Ethernet）是局域网的事实标准。IEEE 802.3系列。

标准	速率	介质	最大距离
10BASE5	10M	粗同轴	500m
10BASE2	10M	细同轴	185m
10BASE-T	10M	双绞线	100m
100BASE-TX	100M	双绞线(Cat5)	100m
1000BASE-T	1G	双绞线(Cat5e+)	100m
1000BASE-SX	1G	多模光纤	550m
1000BASE-LX	1G	单模光纤	5km

标准	速率	介质	最大距离
10GBASE-T	10G	双绞线(Cat6a)	100m
10GBASE-SR	10G	多模光纤	300m
10GBASE-LR	10G	单模光纤	10km

以太网MAC地址

每个以太网网卡都有一个全球唯一的48位（6字节）MAC地址。

- 前3字节：OUI（组织唯一标识符），由IEEE分配给厂商
- 后3字节：厂商自己分配
- 例：00-1A-2B-3C-4D-5E

广播地址：FF-FF-FF-FF-FF-FF（全1） 多播地址：第一字节最低位为1

以太网MAC帧格式

以太网V2帧格式（最常见）：

```

+-----+-----+-----+-----+-----+
| 目的MAC | 源MAC | 类型   | 数据   | FCS   |
| 6字节   | 6字节   | 2字节   | 46-1500 | 4字节   |
+-----+-----+-----+-----+-----+

```

关键字段：

- 前导码（Preamble）：8字节（7字节同步+1字节帧开始定界符），不算在帧长内。
- 类型/长度：>1536表示上层协议类型（0x0800=IPv4, 0x86DD=IPv6）；≤1500表示数据长度（802.3格式）。
- 数据：46-1500字节。少于46字节用填充（pad）补足。
- FCS：CRC-32校验码。

交换式以太网：从共享到交换

早期的以太网用集线器（Hub），所有端口共享带宽、同一个冲突域。交换机（Switch）每个端口是一个独立的冲突域，全双工，无冲突。

网桥（Bridge）：连接两个LAN，根据MAC地址决定是否转发帧。交换机的本质就是多端口网桥。

自学习算法（逆向学习）

交换机怎么知道哪个MAC地址在哪个端口？自己学！

1. 初始状态：交换机的转发/过滤表为空
2. 交换机收到一帧，记录源MAC地址和入端口的映射关系
3. 查表找目的MAC：
 - 找到了且端口≠入端口 → 从该端口转发

- 没找到 → 向除入端口外的所有端口泛洪（Flooding）

4. 表项有老化时间（300秒），到期删除

小明："所以交换机不用配置，插上就能用——因为它自己会学！"

生成树协议（STP）

为了提高可靠性，网络可能有冗余链路。但冗余链路会形成环路——帧在环里无限转发，产生广播风暴。

STP的作用：在逻辑上把冗余链路阻断，形成一个无环的树状拓扑。主链路断了自动切换到备用链路。

IEEE 802.1D STP工作过程：

1. 选举根桥（Root Bridge）——BID最小的交换机
2. 每个非根桥选一个根端口（到根桥最近的端口）
3. 每个网段选一个指定端口（Designated Port）
4. 其余的端口被阻塞（Blocking），不转发数据

RSTP（IEEE 802.1w，快速生成树）：收敛时间从50秒降到几秒。

VLAN（虚拟局域网）

交换机可以把不同端口划分到不同的VLAN，将一个物理交换机变成多个逻辑上隔离的交换机。

[Diagram: 一台交换机，端口1-4属于VLAN10（市场部），端口5-8属于VLAN20（技术部）。VLAN10和VLAN20之间不直接通信——必须通过路由器或三层交换机]

VLAN帧格式（IEEE 802.1Q）：在以太网帧中插入4字节的802.1Q标签，包含：

- TPID（0x8100，标识这是802.1Q帧）
- PCP（优先级，3位）
- CFI（规范格式标识）
- VID（VLAN ID，12位）：支持4094个VLAN（0和4095保留）

Trunk链路：交换机之间的链路，传输带802.1Q标签的帧（多个VLAN混合传输）。

THERE ARE NO DUMB QUESTIONS

Q: 交换机和路由器有什么区别？ A: 交换机工作在数据链路层（看MAC地址），路由器工作在网络层（看IP地址）。但"三层交换机"模糊了这条线——它有路由功能。

Q: VLAN之间怎么通信？ A: 必须通过路由器或三层交换机的路由功能。不能直接跨VLAN通信——这就是VLAN隔离的意义。

BULLET POINTS

- 数据链路层把不可靠的物理链路变成可靠的逻辑链路，负责成帧、差错检测、流量控制。
- 成帧方法：字符计数法（脆弱）、字符填充法（DLE标记）、比特填充法（01111110标志，零比特填充）。
- CRC循环冗余检验是主要的差错检测方法。发送方在数据后附加余数（校验码），接收方做同样的除法验证余数为0。

- 停止等待协议：发一帧等一确认，简单但信道利用率低（尤其高延迟链路）。
- 后退N帧（GBN）：连续发送，累积确认，出错后从丢失帧开始重传所有后续帧。窗口 $\leq 2^n - 1$ 。
- 选择重传（SR）：只重传出错帧，接收方缓存乱序帧。效率最高但接收方需缓冲区。
- PPP是点对点链路层协议。使用LCP建立链路、可选的PAP/CHAP认证、NCP配置网络层。
- ALOHA（纯/时隙）是最早的MAC协议。CSMA/CD（先听后发+边发边听+冲突退避）是以太网的MAC机制。CSMA/CA（RTS/CTS/ACK）是Wi-Fi的MAC机制。
- 以太网最小帧长64字节，最大帧长1518字节（不含前导码）。MAC地址48位全球唯一。交换机通过自学习算法建立MAC地址表。
- STP阻断冗余链路防止环路，RSTP加快收敛。VLAN（802.1Q）实现逻辑隔离，12位VID支持4094个VLAN。

第4章. 网络层：找到你在世界的哪个角落

链路层只负责一跳。网络层负责全局——把你从世界的任何一个角落，路由到任何一个角落。IP地址就是你的"网络身份证"。在这一章，小明要掌握IP地址的奥秘、子网划分的技巧、路由算法的工作原理，还有IPv6这个大救星。

故事：小明要给全国连锁咖啡店组网

小明咖啡店扩张了！北京、上海、广州、成都都有分店。每个店有自己的局域网，但总部的服务器在北京。小明需要让全国的门店通过Internet互通——视频监控、库存系统、收银数据全部联网。

"每个店需要什么IP地址？子网怎么划分？数据走哪条路？" —— 小明发现自己需要网络层的全部知识。

网络层的基本功能

网络层提供主机到主机的通信服务。核心功能：

1. 路由选择（Routing）：决定分组从源到目的的路径
2. 分组转发（Forwarding）：根据路由表将分组从输入端口移到输出端口
3. 拥塞控制（Congestion Control）

网络层提供两种服务：

- 面向连接服务：虚电路（VC）。先建立路径，再传数据。
- 无连接服务：数据报（Datagram）。IP协议就是无连接的。

IPv4地址：32位的网络身份证

IPv4地址是32位二进制数，写为点分十进制：192.168.1.1

分类编址（Classful Addressing）

类别	前缀	网络号	主机号	范围	默认掩码
A类	0	8位	24位	1.0.0.0 - 126.255.255.255	255.0.0.0 (/8)
B类	10	16位	16位	128.0.0.0 - 191.255.255.255	255.255.0.0 (/16)

类别	前缀	网络号	主机号	范围	默认掩码
C类	110	24位	8位	192.0.0.0 - 223.255.255.255	255.255.255.0 (/24)
D类	1110	-	-	224.0.0.0 - 239.255.255.255	多播
E类	1111	-	-	240.0.0.0 - 255.255.255.255	保留

特殊地址：

- 0.0.0.0：本网络本主机（DHCP初始状态）
- 127.x.x.x：环回地址（127.0.0.1，自己发给自己）
- 169.254.x.x：链路本地地址（DHCP失败时自动获取）
- 255.255.255.255：受限广播

THERE ARE NO DUMB QUESTIONS

Q: 为什么没有A类地址128.x.x.x？A: 127.x.x.x是环回地址（A类最后一个网络号127被保留了），所以A类范围是1-126。128开始是B类。

Q: 分类编址现在还用吗？A: 基本不用了。被CIDR（无分类编址）取代。但理解分类编址是理解CIDR的基础。

子网划分（Subnetting）：向主机号"借"位

分类编址太浪费了！一个B类网络65536个IP，谁用得了那么多？

子网划分：把主机号的一部分拿出来当子网号。本质上在两级IP结构中插入第三级：

网络号 | 子网号 | 主机号

假设你有一个C类网络 200.1.1.0/24，想分成4个子网。

4个子网需要2位（ $2^2=4$ ），所以从8位主机号中借2位：

- 子网掩码从 /24 变成 /26（255.255.255.192）
- 每个子网有 $2^6 - 2 = 62$ 个可用地址（减2是因为全0是网络地址、全1是广播地址）

子网1：200.1.1.0/26 （子网号=00）可用：200.1.1.1 - 200.1.1.62，广播：200.1.1.63
 子网2：200.1.1.64/26 （子网号=01）可用：200.1.1.65 - 200.1.1.126，广播：200.1.1.127
 子网3：200.1.1.128/26 （子网号=10）可用：200.1.1.129 - 200.1.1.190，广播：200.1.1.191
 子网4：200.1.1.192/26 （子网号=11）可用：200.1.1.193 - 200.1.1.254，广播：200.1.1.255

[Diagram: 子网划分示意图——原始/24网络，从主机号中借2位变成/26，生成4个子网]

SHARPEN YOUR PENCIL

某公司获得网络地址 172.16.0.0/16（B类）。需要分配以下部门：

- 总部：需要500个主机IP
- 技术部：需要200个主机IP
- 市场部：需要100个主机IP
- 分公司A：需要50个主机IP
- 分公司B：需要50个主机IP

请使用VLSM，从大到小分配子网。写出每个部门的网络地址、子网掩码、可用IP范围。

(提示：500个IP需要 $512=2^9$ ，主机位9，掩码/23。200个需要 $256=2^8$ ，掩码/24。100个需要 $128=2^7$ ，掩码/25。50个需要 $64=2^6$ ，掩码/26。)

SHARPEN YOUR PENCIL SOLUTION

总部 (500个IP, $512=2^9$, 掩码/23) : 网络: 172.16.0.0/23 (掩码255.255.254.0) 范围: 172.16.0.1 - 172.16.1.254, 广播172.16.1.255

技术部 (200个IP, $256=2^8$, 掩码/24) : 网络: 172.16.2.0/24 (掩码255.255.255.0) 范围: 172.16.2.1 - 172.16.2.254

市场部 (100个IP, $128=2^7$, 掩码/25) : 网络: 172.16.3.0/25 (掩码255.255.255.128) 范围: 172.16.3.1 - 172.16.3.126

分公司A (50个IP, $64=2^6$, 掩码/26) : 网络: 172.16.3.128/26 (掩码255.255.255.192) 范围: 172.16.3.129 - 172.16.3.190

分公司B (50个IP) : 网络: 172.16.3.192/26 (掩码255.255.255.192) 范围: 172.16.3.193 - 172.16.3.254

WATCH IT!

计算可用地址数时，记得减去2（网络地址全0和广播地址全1不可用）。/26的每个子网有64个地址，但只有62个可用！

CIDR（无分类域间路由）

分类编址的B类网络快用完了，但还有大量C类没人用。CIDR来了——不再按类别分配，而是按需分配任意长度的前缀。

斜线记法：192.168.1.0/24 —— /24表示前24位是网络前缀。

CIDR的核心概念：超网（Supernetting）——把多个连续的小网络聚合成一个大的CIDR块，减少路由表条目。这叫路由聚合（Route Aggregation）。

例：200.1.0.0/24, 200.1.1.0/24, 200.1.2.0/24, 200.1.3.0/24 可聚合为 200.1.0.0/22

NAT（网络地址转换）

小明："IPv4地址总共才43亿个，全世界这么多人怎么够用？"

NAT是IPv4续命的关键技术。私有地址通过NAT路由器共享一个公有IP上网。

私有地址范围（RFC 1918）：

- 10.0.0.0/8
- 172.16.0.0/12
- 192.168.0.0/16

NAT工作方式：内网私有IP访问Internet时，NAT路由器把源IP（私有）+源端口替换为自己公有IP+新端口。回复到达时反向替换。

GEEK NOTE: NAT虽然解决了地址短缺问题，但也破坏了互联网的"端到端"原则。IPv6就是为了恢复端到端而生的。

IP协议：分组格式

IPv4分组格式

关键字段：

- 版本：4位，IPv4=4
- 首部长：4位，单位4字节。最小5（20字节），最大15（60字节）
- 总长度：16位，首部+数据字节数，最大65535
- 标识：16位，同一分组各分片共享
- 标志：DF（不分片，1=禁止分片），MF（更多分片，1=还有后续）
- 片偏移：13位，单位8字节。该分片在原分组中的位置
- TTL：8位，每跳减1，到0丢弃
- 协议：8位，6=TCP，17=UDP，1=ICMP
- 首部校验和：16位，只校验首部，每跳重新计算

WATCH IT!

片偏移的单位是8字节，不是1字节！偏移185意味着实际偏移 $185 \times 8 = 1480$ 字节。考试常有人直接用1480——注意单位。

分片与重组

当IP分组比链路MTU大时需要分片。

例：4000字节的分组（首部20+数据3980），经过MTU=1500链路。

- 第一片：首部20+数据1480，片偏移=0，MF=1
- 第二片：首部20+数据1480，片偏移=1480/8=185，MF=1
- 第三片：首部20+数据1020，片偏移=2960/8=370，MF=0

SHARPEN YOUR PENCIL

一个3000字节的IP数据报（首部20B + 数据2980B），经过MTU=800B的链路。请计算分片。每个分片数据最多=800-20=780B，取8的倍数=768B。

第1片：数据__字节，片偏移__，MF=__ 第2片：数据__字节，片偏移__，MF=__ 第3片：数据__字节，片偏移__，MF=__ 第4片：数据__字节，片偏移__，MF=__

SHARPEN YOUR PENCIL SOLUTION

$2980 / 776 = 3.84 \rightarrow$ 4片 第1片：776B，片偏移0，MF=1 第2片：776B，片偏移776/8=97，MF=1 第3片：776B，片偏移1552/8=194，MF=1 第4片：652B，片偏移2328/8=291，MF=0 验证：
 $776+776+776+652=2980 \checkmark$

IPv6：更大的地址空间

IPv6把地址从32位扩展到128位。

表示：8组十六进制，冒号分隔。如 2001:0db8:85a3:0000:0000:8a2e:0370:7334 可压缩：
2001:db8:85a3::8a2e:370:7334（连续全0用::，只能用一次）

IPv6分组：固定40字节基本首部，简化很多：

- 无分片（要求发送方做Path MTU Discovery）
- 无首部检验和
- 内置IPsec支持
- 128位地址空间

IPv4向IPv6过渡：

1. 双协议栈（Dual Stack）：同时支持IPv4和IPv6
2. 隧道技术（Tunneling）：IPv6封装在IPv4中穿行
3. NAT64：IPv6-only设备访问IPv4资源

地址解析

ARP（地址解析协议）

问题：我知道对方IP，但以太网需要MAC地址。怎么IP → MAC？

ARP：同一链路上的IP → MAC查询。

1. 主机A查ARP缓存（无命中）
2. 广播ARP请求："谁的IP是X？告诉我你的MAC"
3. IP为X的主机单播ARP响应："我的MAC是M"
4. 主机A存入缓存，封装帧发出

DHCP（动态主机配置协议）

让设备自动获取IP。四步过程：

1. DHCPDISCOVER：客户端广播——"有人给我IP吗？"
2. DHCPOFFER：服务器单播——"用这个IP"
3. DHCPREQUEST：客户端广播——"我要用这个了！"
4. DHCPACK：服务器确认——"OK，归你了"

ICMP与IGMP

ICMP（Internet控制报文协议）

封装在IP中（协议号=1）。两类报文：

差错报告：目的不可达（类型3）、超时（类型11，traceroute利用）、参数问题（类型12）

询问报文：回送请求/应答（类型8/0，ping）、时间戳请求/应答（类型13/14）

ping原理：发ICMP Echo Request，收Echo Reply，计算RTT。

traceroute原理：TTL递增，利用超时报文获取每跳IP。

IGMP（Internet组管理协议）

管理IP多播组成员。主机用IGMP加入/离开多播组。路由器用IGMP定期查询。

路由选择

RIP协议（距离向量算法）

RIP是最简单的内部网关协议（IGP）。

核心思想：每个路由器维护到各目的网络的距离（跳数），周期性（30秒）把整张路由表发给邻居。邻居比较——邻居距离+1更短就更新。

Bellman-Ford公式： $\text{Distance}(A, D) = \min[\text{Cost}(A, X) + \text{Distance}(X, D)]$

特点：

- 度量：跳数，最大15（16=不可达）
- 慢收敛，无限计数问题
- 解决：水平分割、毒性逆转、触发更新

无穷计数问题：A-B链路断→B设到A不可达→C说"我到A是2跳"→B更新为3→B告诉C→C更新为4→.....数到16。

水平分割：从一个接口学到的路由，不从同一接口发出去。毒性逆转：从某接口学到的路由，从该接口发出时距离设为16。

OSPF协议（链路状态算法）

OSPF是目前最常用的IGP。

核心思想：每个路由器了解全网拓扑，用Dijkstra算法独立计算到所有目的的最优路径。

工作过程：

1. 发现邻居（Hello包）
2. 交换链路状态（LSA泛洪全网）
3. 维护链路状态数据库（LSDB，全网一致）
4. 计算SPF树（Dijkstra）
5. 更新路由表

优点：收敛快、无环路、支持CIDR/VLSM

区域划分：

- 骨干区域（Area 0.0.0.0）：必须连接所有区域
- ABR（区域边界路由器）：连接多个区域
- ASBR（AS边界路由器）：连接OSPF和其他协议

RIP vs OSPF对比：

特性	RIP	OSPF
算法	距离向量	链路状态
度量	跳数 (≤15)	代价 (带宽反比)
收敛	慢 (分钟级)	快 (秒级)
环路	可能	无环路
协议号	UDP 520	IP 89

BGP协议（路径向量算法）

BGP是Internet的"胶水"——连接不同自治系统（AS）。

- eBGP：不同AS之间
- iBGP：同一AS内部

BGP工作原理：

- 通告路径（AS序列），不是距离
- 路径向量防环：路由携带经过的AS列表
- 策略路由：考虑商业策略（LOCAL_PREF、MED等属性）

路由器的结构

两个关键部分：

1. 控制平面（路由选择处理器）：运行路由协议，构建路由表。软件实现（CPU）。
2. 数据平面（转发引擎）：根据转发表线速转发。硬件实现（ASIC/TCAM）。

[Diagram: 路由器结构——输入端口 → 交换结构 → 输出端口。路由选择处理器在上方更新转发表]

IP多播

一对多通信：D类地址224.0.0.0/4。IGMP管理组成员。

- PIM-DM（稠密模式）：先泛洪再剪枝
- PIM-SM（稀疏模式）：显式加入，有汇聚点（RP）

BE THE ROUTER

下面拓扑用RIP（跳数），所有链路代价=1。请填写R1收敛后的路由表。

```

R1  ---- R2  ---- R3
 \              /
  +----- R4 -----+

```

R1连LAN:10.1.0.0/16, R3连LAN:10.3.0.0/16, R4连LAN:10.4.0.0/16

目的网络	下一跳	跳数
10.1.0.0/16	直连	0
10.3.0.0/16	——	——
10.4.0.0/16	——	——

BE THE ROUTER 答案

目的网络	下一跳	跳数
10.1.0.0/16	直连	0
10.3.0.0/16	R2或R4	2
10.4.0.0/16	R4	1

到10.3有两条等价路径 (R1 → R2 → R3, R1 → R4 → R3)，跳数均为2。

BULLET POINTS

- 网络层提供主机到主机的通信，核心功能是路由选择和分组转发。IP协议提供无连接数据报服务。
- IPv4地址32位。分类编址：A(/8)B(/16)C(/24)D(多播)E(保留)。CIDR用斜线记法(/n)取代分类编址，支持路由聚合。
- 子网划分：从主机号借位扩展前缀。VLSM允许不同子网不同掩码。可用地址=2^(主机位)-2。
- NAT让私有网络共享公有IP。私有地址：10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16。
- IPv4分片时片偏移单位是8字节。MTU决定是否分片。IPv6地址128位，无分片，40字节固定首部。
- ARP将IP解析为MAC。DHCP四步获取IP：Discover → Offer → Request → ACK。
- ICMP(ping/traceroute)封装在IP中。IGMP管理多播组成员。
- RIP(距离向量,Bellman-Ford,跳数≤15)有无穷计数问题，用水平分割/毒性逆转缓解。
- OSPF(链路状态,Dijkstra,全网拓扑)收敛快无环路，划分区域(骨干Area0+常规区域)。
- BGP(路径向量,AS路径)是Internet域间路由协议，考虑策略路由。
- 路由器：控制平面(CPU)计算路由表，数据平面(ASIC)线速转发。

第5章. 运输层：可靠的端到端快递

网络层把数据包送到了主机。但主机上同时跑着微信、浏览器、游戏.....数据包该给哪个程序？这就是运输层的工作——把数据从一台主机上的一个进程，送到另一台主机上的一个进程。

运输层有两个快递员：UDP（随便丢门口，快但不负责）和TCP（必须签收，慢但可靠）。在这一章，小明要彻底搞懂TCP——三次握手、四次挥手、滑动窗口、拥塞控制.....这是考试的重头戏！

故事：小明咖啡店的在线点餐系统

小明的咖啡店上线了在线点餐系统。客户用手机小程序点单，订单数据通过网络传到店里的打印机。

但问题来了：有时候订单重复打印，有时候丢单，有时候网络突然变慢。

老张："你这是运输层的典型问题。来，咱们好好捋一捋TCP和UDP。"

运输层概述

运输层是端到端的——只关心源和目标主机上的进程之间怎么通信。

核心功能：

- 多路复用/分用：通过端口号区分不同应用进程
- 可靠传输（TCP）：确认、重传、排序
- 流量控制（TCP）：防止发送方压倒接收方
- 拥塞控制（TCP）：防止发送方压倒网络

端口（Port）

16位整数（0-65535），标识主机上的进程。

范围	类型	例子
0-1023	熟知端口	HTTP:80, DNS:53, SMTP:25, FTP:21/20, SSH:22
1024-49151	注册端口	应用默认端口
49152-65535	动态/私有	客户端临时使用

IP地址 + 端口号 = 套接字（Socket），唯一标识网络上的一个通信端点。

THERE ARE NO DUMB QUESTIONS

Q: 为什么DNS用UDP而不是TCP？ A: DNS查询通常很短（一个请求+一个响应包），用TCP三次握手的开销太大。但DNS区域传输和超过512字节的响应会切换到TCP。53端口同时支持UDP和TCP。

UDP：快如闪电但不管死活

UDP几乎就是"IP+端口号"——没有连接、没有确认、没有流量控制。

UDP特点：

- 无连接：直接发送
- 不可靠：不保证送达、不保证不重复、不乱序
- 面向报文：应用层下来的报文原样发送
- 头部开销小：仅8字节
- 无拥塞控制

UDP报文格式：

- 源端口(16位) + 目的端口(16位)
- 长度(16位) + 检验和(16位)

UDP典型应用：DNS、DHCP、SNMP、RIP、VoIP、在线游戏、TFTP

BRAIN POWER

UDP没有拥塞控制。如果大量UDP流量涌入导致拥塞，TCP会减速，UDP不会——这公平吗？提示：QUIC协议（HTTP/3的底层）是基于UDP但加入了拥塞控制的。这说明什么？

TCP：可靠的字节流

TCP是互联网最重要的运输层协议。HTTP、FTP、SMTP、SSH都建立于TCP之上。

TCP特点：

- 面向连接：三次握手
- 可靠传输：确认+重传+序号
- 面向字节流：不保持报文边界
- 全双工：双方同时收发
- 流量控制：滑动窗口
- 拥塞控制：慢开始/拥塞避免/快重传/快恢复

TCP报文段格式

关键字段：

- 源端口、目的端口：各16位
- 序号（Seq）：32位，发送方字节编号
- 确认号（Ack）：32位，期待收到的下一个字节号
- 数据偏移：4位，首部长度（单位4字节）
- 标志位：URG、ACK、SYN、FIN、RST、PSH
- 窗口大小：16位，接收方剩余缓冲区

TCP连接管理

三次握手——建立连接

[Diagram: 三次握手——Client → Server: SYN, seq=x ; Server → Client: SYN+ACK, seq=y, ack=x+1 ; Client → Server: ACK, seq=x+1, ack=y+1]

Step 1：客户端 → 服务器

- SYN=1, seq=x（客户端初始序号，随机生成）
- 客户端状态：SYN-SENT

Step 2：服务器 → 客户端

- SYN=1, ACK=1, seq=y, ack=x+1
- 服务器状态：SYN-RCVD
- 服务器为这个半连接分配资源

Step 3：客户端 → 服务器

- ACK=1, seq=x+1, ack=y+1
- 双方状态：ESTABLISHED
- 这一步可以携带数据

WATCH IT!

SYN Flood攻击：攻击者发大量SYN但不完成握手，服务器资源耗尽。防御：SYN Cookie——服务器不分配资源，把信息编码到初始序号y中。

四次挥手——释放连接

[Diagram: 四次挥手——Client → Server: FIN, seq=u ; Server → Client: ACK, ack=u+1 ; Server → Client: FIN, seq=v, ack=u+1 ; Client → Server: ACK, ack=v+1]

Step 1：客户端 → 服务器

- FIN=1, seq=u
- 客户端状态：FIN-WAIT-1

Step 2：服务器 → 客户端

- ACK=1, ack=u+1
- 服务器状态：CLOSE-WAIT
- 客户端状态：FIN-WAIT-2

Step 3：服务器 → 客户端

- FIN=1, seq=v, ack=u+1
- 服务器状态：LAST-ACK

Step 4：客户端 → 服务器

- ACK=1, ack=v+1
- 客户端状态：TIME-WAIT（等待2MSL≈2分钟）
- 服务器收到ACK后：CLOSED

为什么要等2MSL？

1. 最后的ACK可能丢失——服务器会重发FIN
2. 让本次连接的所有"迷路"分组从网络中消失

SHARPEN YOUR PENCIL

客户端ISN_C=200，服务器ISN_S=100。写三次握手各步：

第一步（Client → Server）：SYN=__, seq=__, ack=__ 第二步（Server → Client）：SYN=__, ACK=__, seq=__, ack=__ 第三步（Client → Server）：ACK=__, seq=__, ack=__

SHARPEN YOUR PENCIL SOLUTION

第一步：SYN=1, seq=200, ack=0 第二步：SYN=1, ACK=1, seq=100, ack=201 第三步：ACK=1, seq=201, ack=101

TCP可靠传输

确认机制

累积确认：ACK n 表示"前面n-1字节都收到了，期望收到第n字节"。

超时重传

每发一个报文段，启动重传定时器。超时未收到ACK，重传。

RTO（超时时间）估计（Jacobson算法）：

- $SRTT = (1-\alpha)SRTT + \alpha \text{采样RTT}$ ($\alpha=1/8$)
- $RTTVAR = (1-\beta)RTTVAR + \beta |\text{采样RTT} - SRTT|$ ($\beta=1/4$)
- $RTO = SRTT + 4*RTTVAR$

快速重传

收到3个重复ACK → 不等超时，立即重传丢失的报文段。

```
发送方：seg1(seq=1), seg2(seq=101), seg3(seq=201), seg4(seq=301)
接收方：收到seg1→ack=101
          收到seg3(乱序)→ack=101(重复!)
          收到seg4(乱序)→ack=101(重复!)
发送方：3个重复ack=101→立即重传seg2
```

TCP流量控制

接收方处理速度可能比发送方慢。TCP用滑动窗口实现流量控制。

接收方窗口（rwnd）：通过ACK中的"窗口大小"字段告知发送方。

发送方窗口 = min(拥塞窗口cwnd, 接收方窗口rwnd)

[Diagram: 滑动窗口——左侧"已发已确认"，中间"已发未确认（窗口内）"，右侧"可发未发"，最右"不可发"]

零窗口探测：接收方窗口=0时发送方停止发送，但周期发探测报文段防止死锁。

TCP拥塞控制

流量控制是端到端的，但网络中间的路由器也可能被灌死——这就是拥塞。

TCP拥塞控制四个核心算法：

1. 慢开始 (Slow Start)

cwnd从1开始，每收到一个ACK翻倍（指数增长），直到达到sssthresh。

"慢开始"其实不慢！每个RTT翻倍——1,2,4,8,16……指数增长非常快。

2. 拥塞避免 (Congestion Avoidance)

cwnd $\geq$ sssthresh后，进入此阶段。cwnd每RTT线性+1（加性增加）。

3. 快重传 (Fast Retransmit)

收到3个重复ACK → 立即重传丢失报文段。

4. 快恢复 (Fast Recovery)

快重传后无需要降到慢开始：

- $sssthresh = cwnd / 2$
- $cwnd = sssthresh$ （不降到1）
- 进入拥塞避免

AIMD（加性增加乘性减少）

正常：cwnd每RTT+1（加性增加 Additive Increase） 拥塞：cwnd减半（乘性减少 Multiplicative Decrease）

超时 vs 3重复ACK：

- 超时（严重拥塞）→ $sssthresh = cwnd/2$, $cwnd=1$ ，重慢开始
- 3重复ACK（轻度拥塞）→ 快重传+快恢复， $sssthresh = cwnd/2$, $cwnd = sssthresh$

SHARPEN YOUR PENCIL

当前cwnd=16，sssthresh=8，MSS=1000字节。(a) 处于什么阶段？下个RTT后cwnd=? (b) 如果超时，sssthresh和cwnd怎么变？(c) 如果3重复ACK，sssthresh和cwnd怎么变？

SHARPEN YOUR PENCIL SOLUTION

(a) $cwnd=16 \geq 8$ ，拥塞避免。下RTT后 $cwnd=16+1=17$ (b) 超时： $sssthresh=8$ ， $cwnd=1$ （重慢开始）(c) 3重复ACK： $sssthresh=8$ ， $cwnd=8$ （快恢复，直接拥塞避免）

BE THE TCP STACK

初始cwnd=1，sssthresh=16，RTT=100ms，rwnd始终很大。

时间	事件	cwnd值	阶段
t=0	刚建立	1	慢开始
t=1RTT	ACK	2	慢开始
t=2RTT	ACK	4	慢开始
t=3RTT	ACK	8	慢开始

时间	事件	cwnd值	阶段
t=4RTT	ACK	——	——
t=5RTT	ACK	17	拥塞避免
t=6RTT	超时!	——	——
t=7RTT	ACK	——	——

BE THE TCP STACK 答案

t=4RTT: cwnd=16, 拥塞避免（到达sssthresh边界） t=6RTT: 超时 → sssthresh=17/2≈8, cwnd=1, 慢开始
t=7RTT: cwnd=2

BULLET POINTS

- 运输层提供进程到进程的通信。套接字 = IP地址 + 端口号。
- UDP：无连接、不可靠、面向报文、8字节头部、无拥塞控制。用于DNS、DHCP、VoIP等。
- TCP：面向连接、可靠、面向字节流、全双工。三次握手建连，四次挥手释放。
- TCP可靠传输：序号+确认+超时重传。RTO用Jacobson算法估计。3重复ACK触发快重传。
- TCP流量控制：滑动窗口，接收方通过窗口字段告知缓冲区。零窗口时周期探测。
- TCP拥塞控制：慢开始(cwnd指数增长), 拥塞避免(cwnd线性增长), 快重传(3重复ACK), 快恢复(cwnd减半非降1)。
- AIMD：正常cwnd每RTT+1，拥塞cwnd减半。超时 → cwnd=1；3重复ACK → 快恢复。
- TIME-WAIT等待2MSL确保ACK到达和旧分组消失。

第6章. 应用层：网络究竟能做什么

前面五层都是基础设施——现在终于到应用层了，这才是你真正使用的东西。浏览器打开网页、微信发消息、收发邮件、远程登录服务器……一切都靠应用层协议。

在这一章，小明要掌握DNS、HTTP、SMTP/POP3/IMAP、FTP、SSH。应用层协议最多最杂，但每个都有自己的"性格"。

故事：小明咖啡店的线上转型

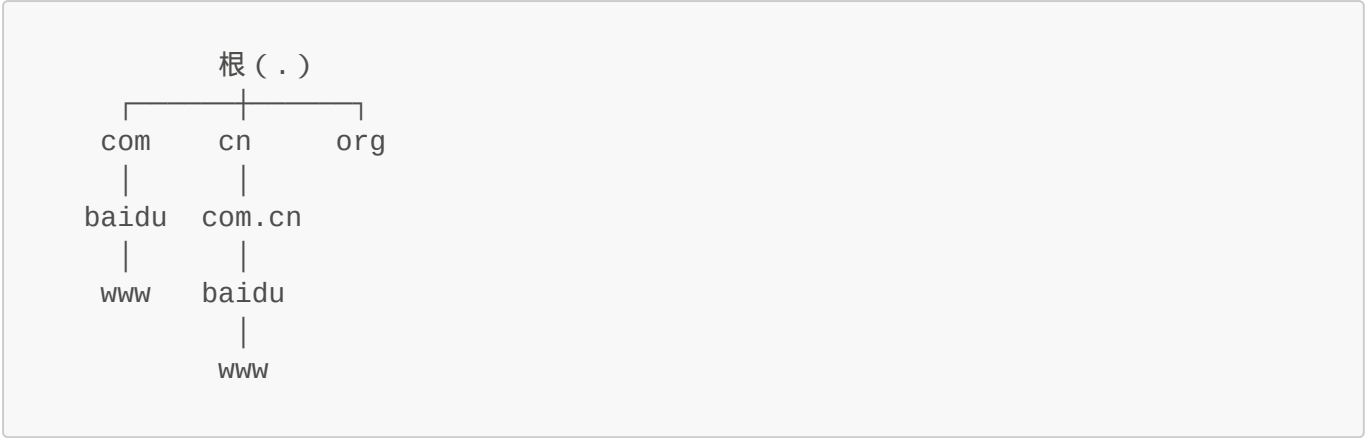
小明上线了官网（www.xiaoming-coffee.com）、邮件系统（order@xiaoming-coffee.com）、FTP服务器给各分店上传日报。但他搞不懂："为什么输入域名就能打开网页？FTP为什么有两个连接？"

老张："这些问题的答案都在应用层。来，我一个一个给你讲。"

DNS：互联网的电话本

DNS（Domain Name System）把域名翻译成IP地址。

域名层次结构



www.baidu.com 完整域名其实是 www.baidu.com (末尾的点代表根，通常省略)。

域名服务器层级

层级	角色
根域名服务器	知道所有TLD服务器IP (全球13组)
顶级域名(TLD)服务器	管理某TLD下的二级域名
权威域名服务器	存储具体域名的解析记录
本地域名服务器	你的ISP的DNS，帮你递归查询

域名解析过程

递归查询：本地DNS替你跑腿，一层一层问，最终返回结果。

迭代查询：本地DNS告诉你去问谁，你自己去问。

实际中：客户端→本地DNS是递归，本地DNS向上是迭代。结果会被缓存（TTL控制缓存时间）。

[Diagram: DNS查询——用户→本地DNS→根DNS→.com TLD DNS→baidu.com权威DNS→返回IP]

DNS记录类型

类型	含义	示例
A	IPv4地址	www A 192.168.1.1
AAAA	IPv6地址	www AAAA 2001:db8::1
CNAME	别名	shop CNAME www
MX	邮件交换	xm.com MX mail.xm.com
NS	该域DNS服务器	xm.com NS dns.xm.com
SOA	授权起点	域管理信息
PTR	反向解析(IP→域名)	1.1.168.192.in-addr.arpa PTR www
TXT	任意文本	SPF、DKIM验证记录

小明注册了xiaoming-coffee.com。网站IP: 123.45.67.89。邮件服务器mail.xm.com: 123.45.67.90。shop.xm.com也指向网站。

MX记录必须指向域名，不能直接指向IP！

42 / 51

HTTP状态码

范围	含义	常用例子
1xx	信息	100 Continue
2xx	成功	200 OK, 201 Created, 204 No Content
3xx	重定向	301永久, 302临时, 304 Not Modified
4xx	客户端错误	400 Bad Req, 401未认证, 403禁止, 404未找到
5xx	服务器错误	500内部错误, 502 Bad Gateway, 503不可用

THERE ARE NO DUMB QUESTIONS

Q: 301和302有什么区别？ A: 301永久重定向——浏览器记住新地址。302临时跳转——每次仍访问旧地址。SEO用301告知搜索引擎"搬家了"。

Q: GET和POST有何本质区别？ A: GET是幂等的（多次请求不产生额外效果），用于获取。POST用于提交，产生副作用。GET参数在URL中（有长度限制），POST在实体体中。

Q: HTTP/1.0、1.1、2的主要区别？ A: 1.0每次请求新连接；1.1支持持久连接（Keep-Alive）、管道化；2支持多路复用（一个连接同时多请求）、头部压缩、服务器推送。HTTP/3基于QUIC（UDP）。

HTTPS与SSL/TLS

HTTPS = HTTP + SSL/TLS加密

SSL/TLS工作过程（简化）：

1. 客户端发ClientHello（支持的加密套件）
2. 服务器回ServerHello + 证书
3. 客户端验证证书
4. 双方协商会话密钥（非对称加密交换，对称加密通信）
5. 后续数据用会话密钥对称加密

HTTPS默认端口443，HTTP默认端口80。

电子邮件

系统组成

- 用户代理（UA）：Outlook、网页邮箱
- 邮件服务器：邮局
- 协议：SMTP（发）、POP3/IMAP（收）

SMTP（Simple Mail Transfer Protocol）

发邮件的协议，TCP端口25（加密587/465）。

特点：推协议（Push），客户端→服务器，服务器→服务器。只能传7位ASCII文本（MIME扩展后支持多媒体）。

POP3（Post Office Protocol v3）

收邮件协议，TCP端口110（加密995）。

离线模式：把邮件从服务器下载到本地，服务器上可保留或删除。简单但多设备同步差。

IMAP（Internet Message Access Protocol）

收邮件协议，TCP端口143（加密993）。

在线模式：邮件始终在服务器，客户端只是查看。支持多设备同步、部分下载、服务端文件夹管理。比POP3更现代。

MIME（Multipurpose Internet Mail Extensions）

扩展了电子邮件的功能：支持非ASCII文本、附件、多媒体内容。定义了Content-Type、Content-Transfer-Encoding等头部字段。

文件传输协议FTP

FTP用两个TCP连接：

- 控制连接（端口21）：传命令和响应，整个会话保持
- 数据连接（端口20）：实际传文件数据，每次传输新建

两种工作模式：

- 主动模式（PORT）：服务器主动从20→客户端指定端口
- 被动模式（PASV）：客户端从随机端口→服务器随机端口（更防火墙友好）

TFTP（Trivial FTP）：基于UDP的简单文件传输，端口69。无认证、无目录浏览。用于无盘工作站引导等简单场景。

远程登录：TELNET与SSH

TELNET

TCP端口23。明文传输（包括密码！），不安全，基本被SSH取代。

SSH（Secure Shell）

TCP端口22。加密通信，提供：

- 安全远程登录（代替TELNET）
- 端口转发/隧道
- 安全文件传输（SCP/SFTP）

基于公钥密码进行身份认证（比密码更安全）。

BULLET POINTS

- DNS将域名翻译为IP。层次结构：根→TLD→权威→本地。递归/迭代查询。记录类型：A/AAAA/CNAME/MX/NS/SOA/PTR/TXT。
- URL = 协议+主机+端口+路径+查询+片段。HTTP是无状态请求-响应协议，状态码1xx/2xx/3xx/4xx/5xx。
- HTTP/1.1持久连接和管道化，HTTP/2多路复用，HTTP/3基于QUIC(UDP)。HTTPS通过SSL/TLS加密。
- 电子邮件：SMTP(发,25)、POP3(收,110,离线)、IMAP(收,143,在线)。MIME扩展多媒体支持。
- FTP：控制连接(21)+数据连接(20/随机)。主动/被动模式。TFTP基于UDP(69)。
- SSH加密远程登录(端口22)，替代不安全的TELNET(端口23)。

第7章. 计算机网络安全：守住你的数字城堡

网络连接了世界，但也打开了潘多拉魔盒——黑客、病毒、中间人攻击、DDoS.....没有安全感？别怕。在这一章，小明要学网络安全的三大基石：加密、认证、防火墙。搞定这些，你的数字城堡就有了护城河。

故事：小明的咖啡店被"盯上了"

小明发现咖啡店的Wi-Fi被破解，改成了"喝咖啡免费上网"。更糟的是一次POS机数据被拦截，信用卡信息泄露。小明这才意识到：光能上网不够，还要安全！

"密码学、数字签名、防火墙.....这些听起来好难。"小明挠头。

老张："不难。你只要记住三个问题：数据会不会被偷看？（机密性）数据会不会被篡改？（完整性）对方是真是假？（认证）"

网络安全概述

安全威胁

被动攻击（不改变数据）：

- 窃听（Sniffing）：截获通信内容
- 流量分析（Traffic Analysis）：分析通信模式

主动攻击（改变数据或系统状态）：

- 篡改（Modification）：修改消息内容
- 伪造（Masquerade）：冒充他人身份
- 重放（Replay）：重复发送截获的旧消息
- 拒绝服务（DoS/DDoS）：耗尽目标资源使其无法服务

安全服务

五大安全服务：

1. 机密性（Confidentiality）：数据不被未授权方读取

2. 完整性 (Integrity)：数据不被篡改
3. 可用性 (Availability)：系统和数据随时可用
4. 不可否认性 (Non-repudiation)：通信方不能否认自己做过的事
5. 认证 (Authentication)：验证对方身份是真是假

安全机制

加密、数字签名、报文鉴别、访问控制、流量填充、路由控制、公证.....

密码学基础

对称密钥密码

加密和解密用同一把密钥。快、高效、适合大数据。

DES (Data Encryption Standard)：64位分组，56位密钥。已被破解——1998年Deep Crack用56小时暴力破解。

3DES (Triple DES)：三次DES操作，密钥168位。安全但慢。

AES (Advanced Encryption Standard)：当前标准。128位分组，128/192/256位密钥。Rijndael算法。

[Diagram: 对称加密——发送方和接收方共享密钥K。明文 → 加密(K) → 密文 → 解密(K) → 明文。中间人没有K，看不懂密文]

公钥密码 (非对称密码)

加密和解密用不同的密钥：公钥加密，私钥解密。任何人都能用你的公钥加密，只有你有私钥能解密。

RSA (Rivest-Shamir-Adleman)：

- 基于大素数因子分解的困难性
- 密钥长度通常2048或4096位
- 比对称加密慢几个数量级，通常用于加密对称密钥 (混合加密体系)

[Diagram: 公钥加密——发送方用接收方的公钥加密 → 密文 → 只有接收方的私钥能解密 → 明文]

密钥分配

对称密码的问题：密钥怎么安全地传给对方？

- 密钥分配中心 (KDC)：可信第三方，给通信双方分配会话密钥
 - Diffie-Hellman密钥交换：通信双方通过交换公开值，各自计算出相同的共享密钥。中间人只能看到公开值，算不出密钥
-

信息安全技术

数字签名

用私钥签名，用公钥验证。证明：消息的确是你发的，内容没被改过。

流程：

1. 发送方对消息做散列 → H
2. 发送方用私钥加密散列值 → 数字签名S
3. 发送方把(消息+签名S)发给接收方
4. 接收方用发送方的公钥解密S → H1
5. 接收方对消息做同样的散列 → H2
6. $H1 == H2$ → 签名有效（身份正确+内容完整）

[Diagram: 数字签名——消息 → 散列 → 私钥加密 → 签名。接收方：签名 → 公钥解密 → 散列1 vs 消息直接散列 → 散列2，二者相等即验证通过]

报文鉴别

只关心消息有没有被篡改，不关心是谁发的（但通常配合签名使用）。

散列函数（Hash）要求：

- 定长输出（MD5:128位, SHA-1:160位, SHA-2:256/384/512位, SHA-3）
- 单向性：散列值不可退回原消息
- 抗碰撞：找不到两个不同消息有相同散列值

MD5：128位，已被破解，不应用于安全场景。

SHA-1：160位，2017年Google找到碰撞，不再推荐。

SHA-2（SHA-256, SHA-384, SHA-512）：目前主流。SHA-256输出256位。

SHA-3：新一代标准，算法结构与SHA-2不同（Keccak）。

报文鉴别码（MAC - Message Authentication Code）

散列+密钥 = 带密钥的散列

HMAC（Hash-based MAC）：

- $HMAC(密钥, 消息) = H[(密钥 \text{ xor } opad) || H[(密钥 \text{ xor } ipad) || 消息]]$
- 同时验证完整性（散列）和认证（密钥）

数字证书与PKI（公钥基础设施）

问题：我怎么知道这个公钥真的是张三的？万一中间人把自己的公钥冒充成张三的呢？

数字证书：由可信第三方（CA，Certificate Authority）签发，绑定身份和公钥。

X.509证书包含：版本、序列号、签名算法、签发者、有效期、主体（持有者）、主体公钥、扩展字段、CA签名

PKI（公钥基础设施）：

- CA（证书认证机构）：签发证书
- RA（注册机构）：审核申请者身份
- 证书仓库：存储和分发证书
- 证书吊销列表（CRL）：已吊销的证书

验证链：根CA证书(自签名,绝对信任) → 中间CA证书 → 终端实体证书

安全协议

IPSec (IP安全)

在网络层提供安全服务。两种模式：

- 传输模式：只加密IP载荷（端到端）
- 隧道模式：加密整个IP分组（用于VPN）

两种协议：

- AH (Authentication Header)：只提供认证和完整性（不加密！）
- ESP (Encapsulating Security Payload)：提供认证+加密（最常用）

安全关联 (SA)：通信双方就安全参数达成一致的"合同"。

IKE (Internet Key Exchange)：自动协商和建立SA。

SSL/TLS (传输层安全)

在运输层之上、应用层之下提供安全。HTTPS就是HTTP+TLS。

TLS握手协议：协商版本、加密套件、交换密钥、证书验证。

TLS记录协议：用协商好的密钥加密实际传输的应用数据。

防火墙技术

防火墙在网络边界检查和控制进出流量。

分组过滤 (Packet Filtering)：根据IP/端口/协议等规则放行或拒绝。工作在网络层/运输层。

状态检测 (Stateful Inspection)：不仅看单包，还跟踪连接状态。更智能，更安全。

应用网关 (Application Gateway/Proxy)：工作在应用层，检查应用协议内容。安全性最高但性能最低。

入侵检测系统 (IDS)

检测网络中的异常和攻击行为。

- 基于特征 (Signature-based)：匹配已知攻击模式。无法检测新攻击。
- 基于异常 (Anomaly-based)：建立正常行为基线，检测偏离。可能有误报。

IDS (入侵检测)：只报警，不干预。IPS (入侵防御)：报警+自动阻断。

VPN (虚拟专用网)

在公网上建立加密的"私人通道"。

- 站点到站点VPN：连接两个网络（如总部和分公司）
- 远程访问VPN：单个用户远程接入公司网络

常见协议：IPSec VPN、SSL VPN、WireGuard

[Diagram: VPN——公共Internet中间是一条加密隧道。两台内网设备通过隧道通信，仿佛在同一局域网]

BULLET POINTS

- 安全威胁分被动攻击（窃听、流量分析）和主动攻击（篡改、伪造、重放、DoS）。五大安全服务：机密性、完整性、可用性、不可否认性、认证。
- 对称密码（DES/3DES/AES）：同一密钥加解密，快，适合大数据。公钥密码（RSA）：公钥加密私钥解密，慢，通常用于交换对称密钥。
- 数字签名：私钥签名，公钥验证。同时实现认证和完整性。散列函数：MD5/SHA-1/SHA-2/SHA-3，定长输出、单向、抗碰撞。
- 报文鉴别码（MAC/HMAC）：散列+密钥，让只有知道密钥的人才能生成/验证。
- 数字证书（X.509）：CA签名，绑定身份和公钥。PKI体系：CA、RA、证书仓库、CRL。
- IPSec工作在网络层，分AH（认证）和ESP（认证+加密），传输/隧道模式。
- SSL/TLS在运输层之上提供安全。HTTPS=HTTP+TLS。握手协商密钥，记录协议加密数据。
- 防火墙：分组过滤、状态检测、应用网关。IDS检测攻击。VPN在公网建立加密通道。
- SSH提供加密远程登录，TELNET明文不安全，FTP有SFTP加密替代。

第3部分总复习：计算机网络纵横字谜

看看你能不能把第1章到第7章的知识全用上！下面是整个计算机网络部分的纵横字谜。

计算机网络纵横字谜

1		2				3			4		
5				6							
7			8				9				
		10				11					
12				13							

- 运输层：UDP(无连接8字节头) vs TCP(连接/可靠/流)。三次握手+四次挥手。滑动窗口流量控制。慢开始/拥塞避免/快重传/快恢复。
- 应用层：DNS(A/CNAME/MX)。HTTP(状态码+HTTPS+TLS)。SMTP/POP3/IMAP。FTP(双连接)+SSH加密。
- 安全：对称(AES) vs 公钥(RSA)。数字签名+散列+证书PKI。IPSec(网络层)+TLS(运输层)。防火墙+IDS+VPN。

恭喜！你已经完成了计算机网络部分的学习。

从物理层的比特流到应用层的网页浏览，从数据链路层的CRC校验到运输层的TCP拥塞控制——你现在可以自信地对任何人说："我懂计算机网络！"

下一部分见！等考完试，别忘了请小明喝杯咖啡——他的咖啡店现在网络稳得一批。